École Mohammadia d'Ingénieurs

Département Modélisation et Informatique Scientifique

Rapport de Projet de Fin d'Études

Système de Recommandation Publicitaire Multi-Objectif basé sur les Bandits Contextuels Sémantiques

Houbbadi Abderrahim

Encadrant Académique : Madame Asmae El Kassiri

Encadrant Professionnel : Monsieur Fahd Idrissi Khamlichi

Entreprise d'accueil : Devoteam Maroc

Année Universitaire 2025–2026

Contents

1 Contexte, Problématique et État de l'Art **3**

Introduction . . . 3

1.1 Organisme d'Accueil . . . 4

1.2 Publicité Digitale et Google Ads . . . 4

1.2.1 Évolution du Marché Publicitaire . . . 4

1.2.2 L'Écosystème Google Ads . . . 4

1.2.3 Modèles Économiques . . . 5

1.2.4 Le Real-Time Bidding (RTB) . . . 5

1.2.5 Systèmes de Recommandation . . . 6

1.2.6 Recommandation Temps Réel : Principe, Objectifs et Défis . . . 7

1.3 Problématique . . . 8

1.3.1 Le Défi de la Publicité en Temps Réel . . . 8

1.3.2 Gap Technologique . . . 9

1.3.3 Formulation de la Problématique . . . 9

Conclusion du Chapitre . . . 10

2 Modélisation du Système et Cadre Théorique **11**

2.1 Modélisation Mathématique . . . 11

2.1.1 Variables du Modèle . . . 11

2.1.2 Formulation du Problème MOIP . . . 13

2.1.3 Défis de Résolution . . . 14

2.2 État de l'Art . . . 14

2.2.1 Stratégies d'Estimation (Agents) . . . 14

2.2.2 Stratégies d'Optimisation (Politiques MOO) . . . 18

2.3 Approche Hybride de Modélisation . . . 20

2.3.1 Architecture de Décision . . . 20

2.3.2 Approche Candidate : le Bandit Contextuel Sémantique Hybride 21

2.3.3 Validation du Choix de l'Approche . . . 22

3 Implémentation Technique & Architecture de Déploiement **29**

Introduction . . . 29

3.1 Architecture Système MLOps 29
3.1.1 Conception du Pipeline *Closed-Loop* 29
3.1.2 Intégration du Modèle Champion 30
3.1.3 Positionnement Technologique chez Devoteam 30
3.2 Implémentation du Cœur de Décision 31
3.2.1 Le Service de Recommandation 31
3.2.2 L'API REST (FastAPI) 31
3.2.3 L'Abstraction d'Infrastructure (*Factory Pattern*) 32
3.3 Gestion du *Delayed Feedback* (FALCON) 32
3.3.1 Problématique du Biais de Pénalisation 32
3.3.2 Implémentation de l'Approche Inspirée de FALCON 32
3.4 Architecture de Déploiement Cloud-Native 33
3.4.1 Option A : Environnement Local (*Docker Compose*) 33
3.4.2 Option B : Déploiement *Serverless* sur Google Cloud (Production) 33
3.5 Validation de l'Architecture Fonctionnelle 34
Conclusion du Chapitre 34

4 Conception du Système Temps Réel et Évaluation 36
Introduction 36
4.1 Architecture du Pipeline Closed-Loop 36
4.1.1 Conception du Pipeline 36
4.1.2 Le Cœur de Décision : RecommendationService 36
4.1.3 L'API REST (FastAPI) 37
4.1.4 L'Abstraction d'Infrastructure (Factory Pattern) 38
4.2 Gestion du *Delayed Feedback* (FALCON) 38
4.2.1 Problématique du Biais de Pénalisation 38
4.2.2 Implémentation Inspirée de FALCON 38
4.3 Architecture de Déploiement Cloud-Native 39
4.3.1 Environnement Local (Docker Compose) 39
4.3.2 Déploiement Serverless sur GCP (Production) 39
4.4 Validation Temps Réel 39
4.4.1 Test d'Intégration End-to-End 39
4.4.2 Simulation Temps Réel (500 itérations) 40
4.4.3 Test de Charge 42
4.5 Limites et Perspectives 42
Conclusion du Chapitre 43

Conclusion Générale 44

Références Bibliographiques 46

Chapter 1

Contexte, Problématique et État de l'Art

Introduction

La transition vers un modèle publicitaire entièrement algorithmique représente l'une des mutations les plus profondes de l'économie numérique contemporaine. Les décisions qui, il y a encore vingt ans, relevaient d'une négociation humaine entre annonceurs et éditeurs, s'exécutent aujourd'hui en quelques millisecondes, à l'échelle de milliards d'impressions quotidiennes. C'est dans ce contexte de haute intensité informationnelle que s'inscrit le présent projet de fin d'études.

Ce premier chapitre pose les fondements nécessaires à la compréhension du problème traité. Il s'organise autour de trois axes complémentaires. Nous présentons d'abord le cadre institutionnel du projet à travers la description de l'organisme d'accueil, Devoteam Maroc, et son positionnement dans l'écosystème technologique marocain. Nous exposons ensuite les mécanismes fondamentaux de la publicité digitale — de l'évolution du marché à l'architecture du *Real-Time Bidding* (RTB) en passant par les systèmes de recommandation — afin de situer précisément l'apport technologique visé. Enfin, nous articulons la **problématique centrale** de ce travail : la conception d'un système adaptatif capable d'optimiser dynamiquement les performances publicitaires en temps réel, en réponse aux limitations identifiées dans les approches industrielles actuelles.

1.1 Organisme d'Accueil

Note

Je n'ai pas encore rédigé ce paragraphe, vu que je n'ai pas encore le contexte et l'hiérarchie dans lesquels ce projet s'inscrit. Elle doit présenter l'entreprise d'accueil, sa mission, ses activités principales et le contexte dans lequel s'inscrit le stage.

1.2 Publicité Digitale et Google Ads

1.2.1 Évolution du Marché Publicitaire

La publicité digitale a connu une transformation radicale au cours des deux dernières décennies. Le passage des médias traditionnels — presse, télévision, affichage — vers les canaux numériques a engendré un écosystème publicitaire d'une complexité sans précédent. En 2024, les dépenses mondiales en publicité digitale ont dépassé les **600 milliards de dollars** [Interactive Advertising Bureau (IAB) 2024], confirmant la position dominante du numérique dans le paysage médiatique mondial.

Cette évolution s'est structurée en trois phases majeures :

1. **L'ère des bannières statiques (1994–2005)** : les premières publicités en ligne étaient de simples images cliquables, vendues au CPM (*Cost-Per-Mille*) selon un modèle d'achat direct entre annonceurs et éditeurs.

2. **L'ère du Search Marketing (2000–2010)** : avec l'émergence de Google AdWords (devenu Google Ads), le modèle CPC (*Cost-Per-Click*) s'est imposé, alignant le coût publicitaire sur l'engagement réel de l'utilisateur [Goldfarb and Tucker 2011].

3. **L'ère du Programmatic Advertising (2010–présent)** : l'automatisation complète du processus d'achat/vente d'espaces publicitaires via des enchères en temps réel (*Real-Time Bidding*, RTB) a transformé l'industrie en un marché algorithmique [Yuan et al. 2013].

1.2.2 L'Écosystème Google Ads

Google Ads constitue la plateforme publicitaire la plus utilisée au monde, offrant trois canaux principaux :

- **Search Network** : annonces textuelles affichées dans les résultats de recherche Google, ciblées par mots-clés.

- **Display Network** : bannières visuelles diffusées sur un réseau de plus de 2 millions de sites partenaires.
- **YouTube Ads** : publicités vidéo pré-roll, mid-roll et overlay sur la première plateforme vidéo mondiale.

1.2.3 Modèles Économiques

Trois modèles de tarification coexistent dans cet écosystème :

Modèle	Signification	Description
CPC	Cost-Per-Click	L'annonceur paie uniquement lorsqu'un utilisateur clique sur l'annonce.
CPM	Cost-Per-Mille	L'annonceur paie pour chaque 1 000 impressions (affichages).
CPA	Cost-Per-Action	L'annonceur paie uniquement lorsqu'une action définie est réalisée (achat, inscription).

Table 1.1: Modèles de tarification en publicité digitale

1.2.4 Le Real-Time Bidding (RTB)

Le *Real-Time Bidding* (RTB) est le mécanisme central de la publicité programmatique moderne. Son fonctionnement repose sur un processus d'enchères instantanées, illustré ci-dessous :

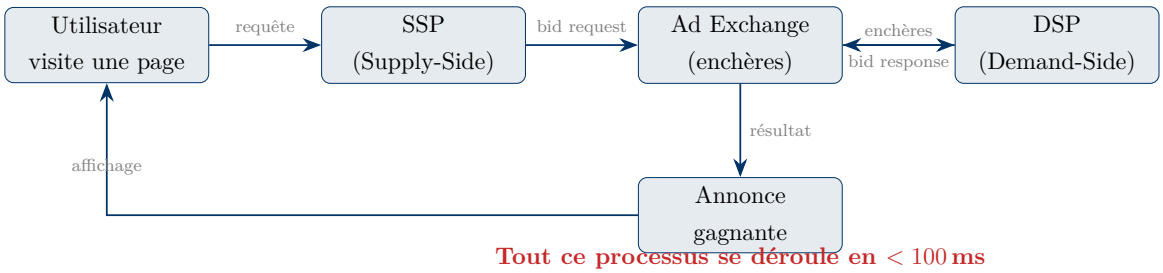

Figure 1.1: Pipeline simplifié du Real-Time Bidding (RTB)

À chaque visite d'un utilisateur sur une page web, une enchère est lancée en temps réel. Les *Demand-Side Platforms* (DSP) des annonceurs soumettent leurs offres, et la meilleure enchère remporte le droit d'afficher sa publicité. Ce processus complet se déroule en moins de **100 millisecondes** [Yuan et al. 2013].

1.2.5 Systèmes de Recommandation

Un **système de recommandation** est un mécanisme de filtrage d'information qui cherche à prédire la préférence ou la note qu'un utilisateur donnerait à un item [Ricci et al. 2015]. Formellement, il s'agit de construire une fonction d'utilité :

$$f : \mathcal{U} \times \mathcal{I} \to \mathbb{R} \tag{1.1}$$

où $\mathcal{U}$ est l'ensemble des utilisateurs et $\mathcal{I}$ l'ensemble des items (dans notre cas, les annonces publicitaires).

Les Trois Familles Classiques

Trois paradigmes fondamentaux structurent le domaine des systèmes de recommandation :

1. **Filtrage collaboratif** (*Collaborative Filtering*) : cette approche se fonde sur l'hypothèse que des utilisateurs ayant manifesté des préférences similaires dans le passé continueront à partager des goûts similaires. La technique phare, la *factorisation de matrice*, projette les utilisateurs et les items dans un espace latent commun de dimension k [Koren et al. 2009] :

$$\hat{r}_{ui} = \mathbf{p}_u^T \mathbf{q}_i \tag{1.2}$$

   où $\mathbf{p}_u \in \mathbb{R}^k$ est le vecteur latent de l'utilisateur u et $\mathbf{q}_i \in \mathbb{R}^k$ celui de l'item i.

2. **Filtrage par contenu** (*Content-Based Filtering*) : cette approche recommande des items similaires à ceux que l'utilisateur a appréciés par le passé, en se fondant sur les attributs intrinsèques des items (catégorie, description textuelle, métadonnées). Elle est robuste au cold-start pour les nouveaux utilisateurs, mais manque souvent de *sérendipité* (capacité à surprendre positivement l'utilisateur).

3. **Approches hybrides** : elles combinent les deux paradigmes précédents pour compenser leurs faiblesses respectives. Par exemple, les systèmes de Netflix exploitent à la fois le filtrage collaboratif (historique de visionnage des utilisateurs similaires) et le filtrage par contenu (genre, réalisateur, acteurs) [Bobadilla et al. 2013].

Transition vers la Recommandation Publicitaire

Dans le contexte publicitaire, les annonces constituent les *items* à recommander. Cependant, deux particularités distinguent la recommandation publicitaire de la recommandation classique :

- **Rotation rapide du catalogue** : contrairement aux films ou aux produits e-commerce, les annonces publicitaires changent fréquemment (création de nouvelles campagnes, expiration d'annonces, etc.).
- **Problème du Cold-Start** : chaque nouvelle annonce entre dans le système sans aucune donnée d'interaction. Le filtrage collaboratif, qui repose sur l'historique des préférences, est structurellement incapable de recommander un item sans historique.

Le *cold-start* désigne l'incapacité d'un système de recommandation à prédire la performance d'un item sans données historiques. En publicité digitale, où les créatifs changent quotidiennement, ce problème est **structurel** et non pas occasionnel.

1.2.6 Recommandation Temps Réel : Principe, Objectifs et Défis

Principe

Dans un système publicitaire en temps réel, le moteur de recommandation doit sélectionner la meilleure annonce pour chaque impression en un temps strictement inférieur à **50 millisecondes** [McMahan et al. 2013]. Au-delà de ce seuil, l'opportunité d'enchère est perdue (*timeout*).

Objectifs Multiples

Le système poursuit simultanément deux objectifs potentiellement contradictoires :

1. **Maximiser l'engagement** (CTR — *Click-Through Rate*) : la probabilité qu'un utilisateur clique sur l'annonce affichée.
2. **Maximiser le revenu** (*Revenue*) : la valeur économique générée, calculée comme le produit du CTR, du taux de conversion (CVR) et du prix par clic (CPC).

Ces deux objectifs sont souvent en conflit : les annonces populaires (fort CTR) tendent à avoir un CPC faible (forte concurrence), tandis que les annonces de niche (faible CTR) offrent un CPC plus élevé [He et al. 2014].

Défis Techniques

La recommandation publicitaire en temps réel soulève au moins cinq défis majeurs :

Défi	**Description**
Latence	Décision en $< 50\,\text{ms}$, excluant les modèles à inférence lente.
Volume	Millions d'impressions par jour nécessitant une scalabilité horizontale.
Exploration-Exploitation	Explorer de nouvelles annonces vs exploiter les meilleures connues.
Non-stationnarité	Le comportement des utilisateurs évolue au cours du temps.
Cold-Start	Nouvelles annonces sans historique de clics.
Multi-objectif	CTR et Revenue sont souvent contradictoires.

Table 1.2: Défis de la recommandation publicitaire en temps réel

[Chapelle et al. 2014] soulignent que la combinaison de ces contraintes disqualifie la majorité des architectures de recommandation académiques, qui opèrent généralement en mode batch (entraînement hors-ligne).

1.3 Problématique

1.3.1 Le Défi de la Publicité en Temps Réel

Le problème de la recommandation publicitaire en temps réel peut être formalisé comme un **problème de décision séquentielle sous incertitude**. À chaque impression t, le système observe un contexte utilisateur $\mathbf{x}_t$ et doit choisir une annonce a_t parmi un ensemble de K candidats. Après avoir affiché l'annonce, le système reçoit un *feedback partiel* : il observe uniquement la récompense de l'annonce choisie, jamais celles des alternatives.

Cette structure correspond exactement au problème de **bandit contextuel** (*contextual bandit problem*), où :

- Le *contexte* $\mathbf{x}_t$ encode le profil de l'utilisateur et le contexte de la page.
- Les *bras* correspondent aux K annonces candidates.
- La *récompense* est multidimensionnelle : $\mathbf{r}_t = (r_t^{\text{click}}, r_t^{\text{rev}}) \in [0,1] \times \mathbb{R}_+$.

Contrairement aux approches supervisées classiques, le système doit **apprendre et s'adapter en continu**, sans phase d'entraînement séparée. Chaque décision est à la fois une exploitation des connaissances acquises et une opportunité d'exploration pour affiner les estimations futures.

1.3.2 Gap Technologique

Les systèmes publicitaires industriels actuels — notamment ceux de Google et Meta — s'appuient sur des modèles de prédiction de CTR massifs basés sur le Deep Learning [He et al. 2014; McMahan et al. 2013]. Cependant, ces systèmes présentent des limitations structurelles :

1. **Dépendance aux données massives** : chaque nouvelle annonce nécessite des milliers d'impressions avant que le modèle ne puisse prédire sa performance de façon fiable.
2. **Cold-start persistant** : l'absence de données historiques pour les nouvelles annonces conduit à des performances dégradées durant la phase d'apprentissage.
3. **Optimisation multi-objectif simpliste** : le compromis CTR/Revenue est généralement traité par une scalarisation fixe (pondération linéaire $w_1 \cdot \text{CTR} + w_2 \cdot \text{Revenue}$), incapable d'explorer les régions non-convexes du front de Pareto.

Gap Identifié

Il n'existe pas, dans la littérature, de cadre unifié intégrant simultanément :

- Des **bandits contextuels** pour la décision temps réel,
- Une **optimisation multi-objectif** rigoureuse (au-delà de la scalarisation simple),
- Un mécanisme de **cold-start sémantique** pour le transfert zéro-shot.

1.3.3 Formulation de la Problématique

Face à ce constat, la problématique centrale de ce projet de fin d'études se décline en trois sous-questions de recherche :

Problématique Centrale

« Comment concevoir un système adaptatif capable d'optimiser dynamiquement les performances publicitaires en temps réel ? »

Cette question se décompose en trois axes :

1. **Modélisation mathématique** : Comment formuler le problème de recommandation publicitaire comme un problème de bandit contextuel multi-objectif ?

2. **Stratégies d'estimation et d'optimisation** : Quels agents d'estimation et quelles politiques d'optimisation multi-objectif (MOO) offrent le meilleur compromis engagement/revenue ?

3. **Approche hybride sémantique** : Comment les approches hybrides intégrant des embeddings sémantiques peuvent-elles résoudre le problème du cold-start par transfert zéro-shot ?

Le chapitre suivant formalisera mathématiquement ce problème et présentera l'état de l'art des méthodes d'estimation et d'optimisation avant d'introduire l'approche hybride proposée.

Conclusion du Chapitre

Ce chapitre a permis de délimiter avec précision le cadre général dans lequel s'inscrit ce projet. Partant de l'analyse de l'organisme d'accueil et de l'écosystème publicitaire digital, nous avons progressivement construit la chaîne de raisonnement menant à la formulation de la problématique centrale.

Trois constats majeurs ressortent de cette analyse :

1. **La complexité croissante du marché publicitaire** impose des systèmes de décision capables d'agir en temps réel, avec une latence inférieure à 100 ms, tout en garantissant la pertinence des annonces pour chaque utilisateur.

2. **Les limites structurelles des approches actuelles** — dépendance aux données massives, cold-start persistant, optimisation multi-objectif par scalarisation fixe — créent un écart technologique (*gap*) qu'aucune solution existante ne comble de manière unifiée.

3. **La problématique formulée** ouvre trois axes de recherche complémentaires : la modélisation mathématique rigoureuse du problème, le benchmark des agents d'estimation et des politiques d'optimisation multi-objectif, et la conception d'une approche hybride sémantique pour le transfert zéro-shot.

Fort de ce diagnostic, le **Chapitre 2** s'attachera à formaliser le problème sous la forme d'un programme d'optimisation multi-objectif à variables entières (MOIP), à dresser un état de l'art exhaustif des méthodes candidates, et à présenter l'approche hybride retenue ainsi que sa validation expérimentale.

Chapter 2

Modélisation du Système et Cadre Théorique

Introduction

Le chapitre précédent a mis en évidence les défis de la recommandation publicitaire en temps réel et l'absence d'un cadre unifié intégrant bandits contextuels, optimisation multi-objectif et cold-start sémantique. Ce chapitre a pour objectif de formaliser mathématiquement ce problème, de présenter l'état de l'art des stratégies d'estimation et d'optimisation, puis d'introduire l'approche hybride sémantique proposée et sa validation expérimentale.

2.1 Modélisation Mathématique

2.1.1 Variables du Modèle

La modélisation du système repose sur deux catégories de variables : les *variables explicatives* (features) qui décrivent le contexte, et les *variables à expliquer* (réponses) que le système cherche à prédire.

Variables Explicatives (Features)

- **Contexte utilisateur** : $\mathbf{x}_u \in \mathbb{R}^d$ — vecteur d'embedding du profil utilisateur ($d = 384$ pour `all-MiniLM-L6-v2`).
  - Encodé à partir de métadonnées textuelles : {âge, sexe, intérêts, historique de navigation}.
  - *Exemples de structures industrielles* :
    - **Criteo** : 13 features numériques (ex: `count_of_views`) + 26 features catégorielles hashées.

    * **Outbrain (OBD)** : `device_type`, `geo_location`, `platform`, `document_id`.
    * **Zozotown (Open Bandit Dataset)** : `age_group`, `gender`, `item_id`.

- **Contexte annonce** : $\mathbf{x}_a \in \mathbb{R}^d$ — vecteur d'embedding de la description de l'annonce.
  - Encodé via SentenceTransformer [Reimers and Gurevych 2019] : titre + description + catégorie → vecteur dense.
- **Contexte global** (agents hybrides) : $\mathbf{x} = [\mathbf{x}_u \| \mathbf{x}_a] \in \mathbb{R}^{2d}$ — concaténation des deux vecteurs.

L'utilisation de `all-MiniLM-L6-v2` [Reimers and Gurevych 2019] comme encodeur sémantique est motivée par sa capacité à capturer la similarité sémantique entre textes tout en maintenant une dimensionnalité raisonnable ($d = 384$). Ce modèle est pré-entraîné sur plus d'un milliard de paires de phrases, garantissant une représentation riche et généralisable.

Les fondements théoriques de cette représentation vectorielle des textes remontent aux travaux de [Mikolov et al. 2013] sur les *word embeddings*, qui ont démontré que des vecteurs denses appris capturent naturellement les relations sémantiques entre mots.

Variables à Expliquer (Réponses)

Le système prédit deux réponses pour chaque paire (utilisateur, annonce) :

1. **Engagement (CTR)** : $y_{\text{click}} \in [0, 1]$ — probabilité de clic.

$$\text{engagement}(u, a) = \sigma(\gamma \cdot \cos(\mathbf{x}_u, \mathbf{x}_a)) \tag{2.1}$$

   où σ est la fonction sigmoïde, $\gamma = 5.0$ est le facteur d'échelle, et $\cos(\cdot, \cdot)$ désigne la similarité cosinus.

2. **Revenue** : $y_{\text{rev}} \in \mathbb{R}_+$ — valeur économique générée.

$$\text{revenue}(u, a) = \underbrace{\text{CTR}(u, a)}_{\text{engagement}} \times \underbrace{\text{CVR}(a)}_{\substack{\text{taux de}\\ \text{conversion}}} \times \underbrace{\text{CPC}(a)}_{\substack{\text{coût par}\\ \text{clic}}} \tag{2.2}$$

Le CVR (*Conversion Rate*) est modélisé comme inversement corrélé au prix : les annonces populaires (CTR élevé) tendent à avoir un taux de conversion plus faible en raison de la saturation du marché. Le CPC (*Cost-Per-Click*) suit un pricing exponentiel inversé : les annonces de niche, moins populaires, commandent un prix par clic plus élevé.

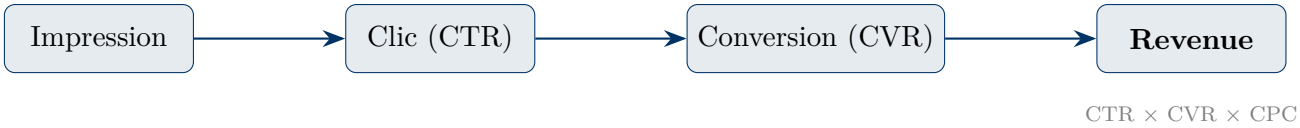

Figure 2.1: Funnel de conversion publicitaire

2.1.2 Formulation du Problème MOIP

Le problème de sélection d'annonces se formalise comme un Programme en Nombres Entiers Multi-Objectifs (*Multi-Objective Integer Programming*, MOIP) :

$$\max_{a \in \{1,\ldots,K\}} \quad \mathbf{f}(a \mid \mathbf{x}_u) = \begin{pmatrix} f_{\text{click}}(a \mid \mathbf{x}_u) \\ f_{\text{rev}}(a \mid \mathbf{x}_u) \end{pmatrix} \tag{2.3}$$

- K = nombre d'annonces actives ($K = 48$ initialement, $K = 60$ après cold-start shock)
- $\mathbf{x}_u$ = contexte utilisateur (vecteur d'embedding)
- $f_{\text{click}}(a \mid \mathbf{x}_u)$ = CTR estimé pour l'annonce a sachant l'utilisateur
- $f_{\text{rev}}(a \mid \mathbf{x}_u)$ = revenue estimé

Contraintes :

1. **Sélection discrète** : exactement un bras sélectionné par itération ($z_k \in \{0, 1\}$, $\sum_k z_k = 1$).
2. **Temps réel** : $t_{\text{décision}} < 50\,\text{ms}$.
3. **Feedback partiel** : seule la récompense de l'annonce choisie est observée (problème de bandit).

Notion de Dominance de Pareto

Dans ce problème multi-objectif, il n'existe généralement pas de solution unique maximisant simultanément tous les objectifs. On fait appel à la notion de **dominance de Pareto** [Deb 2001] :

- L'annonce a *domine* a' (noté $a \succ a'$) si $f_i(a) \geq f_i(a')$ pour tout i et $f_j(a) > f_j(a')$ pour au moins un j.
- L'ensemble des annonces non-dominées forme le **front de Pareto**.
- Il n'existe pas de solution universellement optimale $\rightarrow$ une *politique de décision* est nécessaire.

2.1.3 Défis de Résolution

La résolution du problème P_{MOIP} en contexte de publicité temps réel présente cinq défis fondamentaux :

1. **Dilemme exploration-exploitation** : explorer de nouvelles annonces pour mieux estimer leurs récompenses vs. exploiter les annonces déjà connues comme performantes.
2. **Feedback partiel** (*bandit feedback*) : contrairement à l'apprentissage supervisé, on n'observe **jamais** le reward des annonces non sélectionnées.
3. **Cold-start** : les nouvelles annonces ont zéro historique $\rightarrow$ le modèle classique (per-arm) ne peut pas prédire leur performance sans exploration.
4. **Conflit des objectifs** : les annonces à fort CTR ont souvent un faible CPC (et inversement) $\rightarrow$ le trade-off est non-trivial.
5. **Scalabilité** : K peut atteindre des milliers d'annonces ; le temps de décision doit rester $< 50\,\text{ms}$.

2.2 État de l'Art

2.2.1 Stratégies d'Estimation (Agents)

Le cœur du système réside dans sa capacité à estimer les fonctions $f_{\text{click}}(a \mid \mathbf{x}_u)$ et $f_{\text{rev}}(a \mid \mathbf{x}_u)$ à partir des interactions observées. Nous présentons ici les sept familles d'agents étudiées dans ce projet, regroupées en trois catégories.

A. Bandits Contextuels Linéaires

LinUCB [Li et al. 2010] : LinUCB modélise la récompense de chaque bras k comme un produit scalaire linéaire $\hat{r}_k = \mathbf{x}^T \hat{\boldsymbol{\theta}}_k$, où $\hat{\boldsymbol{\theta}}_k$ est estimé par régression Ridge. L'algorithme maintient pour chaque bras une matrice de covariance $A_k \in \mathbb{R}^{d \times d}$ et un vecteur $\mathbf{b}_k \in \mathbb{R}^d$:

$$\hat{\boldsymbol{\theta}}_k = A_k^{-1} \mathbf{b}_k \tag{2.4}$$

La règle de décision combine exploitation et exploration via une borne de confiance supérieure (UCB) :

$$\text{UCB}_k = \underbrace{\mathbf{x}^T \hat{\boldsymbol{\theta}}_k}_{\text{exploitation}} + \underbrace{\alpha \sqrt{\mathbf{x}^T A_k^{-1} \mathbf{x}}}_{\text{exploration}} \tag{2.5}$$

La mise à jour de A_k^{-1} se fait par la formule de **Sherman-Morrison** en $O(d^2)$ au lieu de l'inversion matricielle directe en $O(d^3)$, ce qui est crucial pour respecter la contrainte de latence.

Thompson Sampling [Agrawal and Goyal 2013] : cette approche bayésienne échantillonne à chaque étape un vecteur de paramètres $\tilde{\boldsymbol{\theta}}_k$ depuis la distribution a posteriori :

$$\tilde{\boldsymbol{\theta}}_k \sim \mathcal{N}\left(\hat{\boldsymbol{\theta}}_k,\ v^2 A_k^{-1}\right) \tag{2.6}$$

Le bras sélectionné est celui qui maximise $\mathbf{x}^T \tilde{\boldsymbol{\theta}}_k$. L'exploration est *stochastique* (par nature de l'échantillonnage), contrairement à l'exploration *déterministe* de LinUCB.

B. Bandits à Réseaux de Neurones

NeuralUCB [Zhou et al. 2020] : NeuralUCB remplace le modèle linéaire par un réseau de neurones $f_{\boldsymbol{\theta}}$ pour capturer les non-linéarités dans la relation contexte-récompense :

$$\mathrm{UCB}_k = f_{\boldsymbol{\theta}}(\mathbf{x}_k) + \alpha\sqrt{\mathbf{g}(\mathbf{x}_k)^T Z^{-1} \mathbf{g}(\mathbf{x}_k)} \tag{2.7}$$

où $\mathbf{g}(\mathbf{x}_k) = \nabla_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}}(\mathbf{x}_k)$ est le gradient du réseau par rapport à ses paramètres, et Z est la matrice de covariance dans l'espace des gradients. Cette approche bénéficie de l'expressivité des réseaux de neurones tout en conservant le principe UCB pour l'exploration.

NeuralTS [Zhang et al. 2021] : version Thompson Sampling de NeuralUCB. L'échantillonnage s'effectue dans l'espace des gradients du réseau, combinant l'expressivité non-linéaire avec l'exploration stochastique bayésienne.

DeepBandit [Riquelme et al. 2018] : un réseau de neurones bayésien complet avec exploration ε-greedy. L'incertitude est mesurée par dropout ou par ensemble de réseaux, offrant une alternative plus stable que l'approche par gradients.

C. Architectures de Déploiement Temps Réel

Les deux architectures suivantes ne sont pas des agents bandit indépendants, mais des **surcouches de déploiement** conçues pour fonctionner *au-dessus* d'un agent UCB existant (LinUCB dans notre implémentation). Elles répondent à des contraintes propres au déploiement temps réel et ont été incluses dans le benchmark pour quantifier leur impact sur les performances.

Offline-to-Online [Agarwal et al. 2014] : cette architecture implémente une stratégie de mise en production progressive. Dans une première phase (*burn-in*), le modèle est initialisé avec un **pessimisme contrôlé** (réduction du terme d'exploration UCB par un facteur de décroissance $\lambda_{\text{decay}} = 0.995$ par itération), simulant le comportement d'un système qui dispose déjà de données offline. Le pessimisme se dissipe progressivement, permettant la transition vers un régime UCB standard. Cette approche est particulièrement utile lors du démarrage à froid d'un nouveau système avec des données historiques disponibles.

Delayed Feedback [Joulani et al. 2013] : cette architecture adresse le problème de la **latence des retours** en publicité, où le signal de récompense (clic, conversion) peut arriver avec un délai de plusieurs heures après l'impression. L'architecture maintient un buffer de feedbacks en attente (`pending deque`) et applique une **correction de biais** sur les récompenses observées avant de les propager au modèle UCB sous-jacent. Le facteur de correction $\delta = 0.8$ compense l'impact statistique des feedbacks non encore reçus.

Agent / Architecture	**Type**	**Complexité Update**	**Exploration**
Agents Bandit			
LinUCB	Linéaire	$O(d^2)$	Déterministe (UCB)
Thompson	Linéaire	$O(d^2)$	Stochastique (posterior)
NeuralUCB	Non-linéaire	$O(p^2)$	Déterministe (UCB gradients)
NeuralTS	Non-linéaire	$O(p^2)$	Stochastique (gradients)
DeepBandit	Non-linéaire	$O(p^2)$	ε-greedy
Architectures de Déploiement (surcouche LinUCB)			
Offline2On	Linéaire	$O(d^2)$	Pessimiste $\rightarrow$ optimiste
DelayedFB	Linéaire	$O(d^2)$	UCB + buffer de feedbacks
Approches LLM Pures (politique sémantique directe)			
LlamaReasoning	LLM (chain-of-thought)	Sans mise à jour	Zero-Shot (raisonnement)
LlamaInstruct	LLM (instruction)	Sans mise à jour	Zero-Shot (prompt direct)

Table 2.1: Agents bandit, architectures de déploiement et approches LLM (d = dimension feature, p = paramètres réseau)

D. Approches LLM Pures (Zero-Shot Policy)

Contrairement aux bandits qui **apprennent** d'une récompense cumulative, les agents LLM agissent comme des **politiques de décision sémantique directe** : ils reçoivent le vecteur de contexte utilisateur $\mathbf{x}_u$ et sélectionnent l'annonce optimale par raisonnement sur le *sens linguistique* du profil, sans paramètres apprenables. Ce paradigme s'inscrit dans les travaux récents sur les LLMs comme rankers zéro-shot [Hou et al.

2024b] et les agents LLM en systèmes de recommandation [Lin et al. 2024].

A. Principe de Fonctionnement. L'agent LLM reçoit un prompt structuré contenant $\mathbf{x}_u$ et le nombre d'annonces disponibles, et retourne directement l'index $k^* = \text{LLM}(\mathbf{x}_u)$ — *la politique MOO n'est jamais appelée.* Deux variantes ont été testées :

- **LlamaReasoning** : prompt avec *chain-of-thought* étendu — le modèle produit une trace de raisonnement avant de sélectionner l'annonce.
- **LlamaInstruct** : prompt d'instruction directe sans raisonnement intermédiaire.

B. Avantage — Compréhension Sémantique Immédiate. Les LLMs ne nécessitent aucune exploration préalable : la performance est exploitable dès $t = 0$. Sur le benchmark, LlamaReasoning atteint eng = 0.648 dès la première itération, comparable à LinUCB classique (0.619) *après convergence*, sans aucun entraînement. Des travaux récents [Fang et al. 2024] montrent que les LLMs zero-shot peuvent surpasser des systèmes collaboratifs supervisés dans certains domaines.

C. Limite 1 — Invariance à la Politique MOO. Le LLM ne peut pas être guidé par une politique ε-Constraint ou NSGA-II : son prior d'entraînement optimise implicitement selon un biais engagement-textuel, sans possibilité d'ajuster le trade-off engagement/revenue. Résultat observé : les scores sont **identiques sur les 10 politiques** — la simulation ne s'exécute qu'une seule fois.

D. Limite 2 — Latence Incompatible avec le RTB. La publicité programmatique impose un délai de réponse < 100ms (protocole OpenRTB [Wu et al. 2024]). Les modèles locaux mesurés dans notre benchmark contredisent structurellement cette contrainte :

Agent	Temps/décision	SLA RTB (<100ms)	Ratio
LlamaReasoning	6 901 ms	Non	×69 000
LlamaInstruct	2 127 ms	Non	×21 000
LinUCB (référence)	15 ms	Oui	×0.15

Table 2.2: Comparaison de la latence de décision : agents LLM vs bandit de référence

La nature probabiliste des LLMs est structurellement incompatible avec les exigences de déterminisme et de vitesse des enchères programmatiques. Des approches hybrides (LLM *offline* pour l'initialisation + bandit *online* pour l'adaptation) sont proposées dans la littérature pour concilier les deux paradigmes [Lin et al. 2024].

E. Rôle dans le Benchmark. *Les agents LLM servent d'oracle sémantique de référence : ils révèlent le plafond de performance atteignable par compréhension contextuelle pure sans apprentissage. Leur écart sur le revenue (rev* $= 0.035$–0.054 *vs rev* $= 0.094$ *pour H-DeepBandit) démontre que la connaissance linguistique seule est insuffisante pour maximiser des objectifs économiques quantifiés — ce qui justifie l'approche hybride bandit sémantique proposée.*

2.2.2 Stratégies d'Optimisation (Politiques MOO)

Une fois les estimations $\hat{\mu}_{t,k}^{\text{CTR}}$ et $\hat{\mu}_{t,k}^{\text{VAL}}$ obtenues par l'agent, une *politique MOO* sélectionne l'annonce qui réalise le meilleur compromis entre les objectifs. Nous présentons les dix politiques étudiées, regroupées en trois catégories.

A. Méthodes Exactes Classiques

Scalarisation Linéaire. La méthode la plus intuitive agrège les objectifs en un score scalaire unique :

$$a^* = \arg \max_{a \in \{1,\ldots,K\}} \sum_{i=1}^{m} w_i \cdot f_i(a \mid \mathbf{x}_u) \tag{2.8}$$

où $\mathbf{w} = (w_1, w_2)$ est un vecteur de poids défini par les priorités métier. Cette méthode est simple et rapide ($O(K)$) mais ne peut atteindre que les solutions Pareto situées sur la partie convexe du front [Marler and Arora 2004].

ε-Contrainte. Cette méthode maximise un objectif principal sous contrainte de satisfaction d'un seuil minimal pour les autres objectifs :

$$\max_{a} \; f_1(a \mid \mathbf{x}_u) \quad \text{sous contrainte} \quad f_2(a \mid \mathbf{x}_u) \geq \varepsilon \tag{2.9}$$

Contrairement à la scalarisation, cette approche est capable d'atteindre *toutes* les solutions Pareto, y compris dans les régions non-convexes du front [Ehrgott 2005].

Pareto-Chebyshev. Cette méthode minimise la distance de Chebyshev (distance L_∞) au point idéal $\mathbf{z}^*$, en se restreignant aux solutions non-dominées :

$$a^* = \arg \min_{a \in \mathcal{P}} \; \max_{i} \left| \frac{f_i(a) - z_i^*}{z_i^* - z_i^{\text{nadir}}} \right| \tag{2.10}$$

où $\mathcal{P}$ est l'ensemble des solutions Pareto-optimales.

B. Méthodes Exactes Avancées

Cinq méthodes exactes avancées complètent l'arsenal d'optimisation :

Méthode	**Principe**
MOBB	*Multi-Objective Branch and Bound* : exploration de l'arbre de décision avec élagage des branches dominées.
TwoPhase	Phase 1 : identification du front de Pareto complet. Phase 2 : sélection de la solution préférée.
OSS	*Optimal Scalarization Search* : recherche de la pondération optimale dans l'espace des poids.
MODP	*Multi-Objective Dynamic Programming* : programmation dynamique adaptée au cas multi-objectif.
MOA*	Extension de l'algorithme A* pour la navigation multi-objectif dans l'espace des solutions.

Table 2.3: Méthodes exactes avancées pour l'optimisation multi-objectif

Ces méthodes garantissent l'optimalité de la solution mais présentent une complexité temporelle exponentielle dans le pire des cas [Ehrgott and Gandibleux 2000]. Elles sont principalement utiles comme *vérité terrain* pour évaluer la qualité des méthodes approximatives.

C. Métaheuristiques

NSGA-II [Deb et al. 2002] : l'algorithme génétique multi-objectif le plus utilisé. Il maintient une population de solutions candidates et applique :

1. **Tri par non-dominance** : classement des solutions par rang de Pareto (Front 1, Front 2, etc.).

2. **Distance de crowding** : favorise la diversité des solutions sur le front pour éviter le regroupement.

MOEA/D [Zhang and Li 2007] : décompose le problème multi-objectif en un ensemble de sous-problèmes scalaires résolus simultanément par une population d'agents coopératifs. Chaque sous-problème utilise un vecteur de poids différent, permettant d'approximer l'ensemble du front de Pareto.

Catégorie	Politique	Principe	Optimalité
Classiques	Scalarisation	Somme pondérée	Convexe seulement
	ε-Contrainte	Objectif + seuil	Complète
	Pareto-Chebyshev	Distance L_∞	Complète
Exactes	MOBB	Branch-and-Bound	Exacte
	TwoPhase	Bi-phasique	Exacte
	OSS	Recherche de poids	Exacte
	MODP	Prog. dynamique	Exacte
	MOA*	A* multi-objectif	Exacte
Méta.	NSGA-II	Algo. génétique	Approximée
	MOEA/D	Décomposition	Approximée

Table 2.4: Synthèse des 10 politiques d'optimisation multi-objectif

2.3 Approche Hybride de Modélisation

2.3.1 Architecture de Décision

Le pipeline de décision complet du système se décompose en cinq étapes séquentielles :

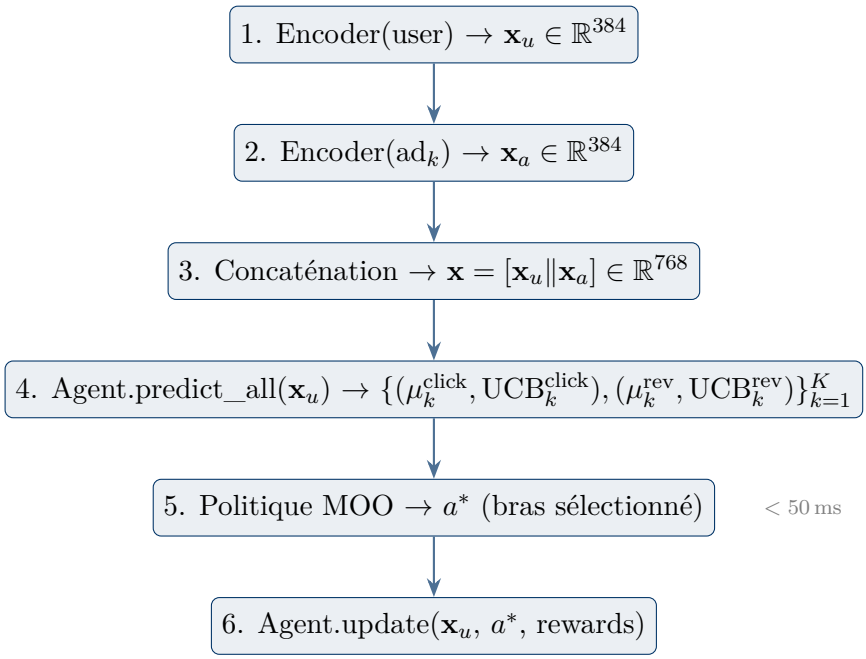

Figure 2.2: Pipeline de décision du système de recommandation

Le choix architectural fondamental de ce projet est la comparaison entre deux architectures de modélisation :

	Classique (Disjoint)	**Hybride (Global Semantic)**
Modèle	K modèles indépendants (un par bras)	Un seul modèle global partagé
Contexte	$\mathbf{x}_u$ (embedding utilisateur seul)	$[\mathbf{x}_u \| \mathbf{x}_a]$ (user + ad embedding)
Cold-start	Impossible (nouveau bras = pas de modèle)	Immédiat (embedding de la nouvelle annonce)
Paramètres	$K \times d^2$ (une matrice A par bras)	$1 \times (2d)^2$ (une seule matrice A)

Table 2.5: Comparaison des architectures classique et hybride

2.3.2 Approche Candidate : le Bandit Contextuel Sémantique Hybride

L'approche hybride sémantique proposée dans ce projet constitue une contribution originale qui s'inspire de deux travaux fondateurs, sans toutefois disposer d'un équivalent direct dans la littérature :

- **LinUCB-Hybrid** [Li et al. 2010] (2010) : a introduit la notion de paramètres partagés entre bras pour un bandit contextuel linéaire, mais avec des features artisanales.
- **SentenceTransformer** [Reimers and Gurevych 2019] (2019) : a rendu possible la représentation sémantique dense de textes arbitraires en vecteurs comparables.

La nouveauté introduite ici est leur combinaison dans un cadre publicitaire bi-objectif avec mécanisme de transfert zéro-shot : un seul modèle global LinUCB opère sur le contexte concaténé $[\mathbf{x}_u \| \mathbf{x}_a]$ où les deux vecteurs sont des embeddings sémantiques, permettant de prédire immédiatement la performance de toute nouvelle annonce sans exploration préalable [Hou et al. 2024a].

L'approche proposée repose sur quatre piliers :

1. Encodage Sémantique

Les profils utilisateurs et les descriptions d'annonces sont encodés en vecteurs denses de dimension $d = 384$ par le modèle `all-MiniLM-L6-v2` [Reimers and Gurevych 2019]. Ce modèle pré-entraîné sur plus d'un milliard de paires de phrases capture la similarité sémantique entre textes via la similarité cosinus.

2. Modèle Global Partagé

Au lieu de maintenir K modèles per-arm indépendants, un seul modèle LinUCB opère sur le contexte concaténé $\mathbf{x} = [\mathbf{x}_u \| \mathbf{x}_a] \in \mathbb{R}^{768}$:

- La matrice de covariance inverse $A^{-1} \in \mathbb{R}^{768\times768}$ est **partagée** entre tous les bras.
- Les vecteurs $\mathbf{b}_{\text{obj}} \in \mathbb{R}^{768}$ sont partagés par objectif.
- La mise à jour via Sherman-Morrison reste en $O(d^2)$.

3. Transfert Zéro-Shot (Zero-Shot Transfer)

C'est l'avantage déterminant de l'architecture globale. Quand une nouvelle annonce apparaît dans le système :

Mécanisme Zero-Shot

1. Son embedding $\mathbf{x}_{a_{\text{new}}}$ est calculé par SentenceTransformer (opération offline, $\approx 10\,\text{ms}$).
2. Le modèle global peut **immédiatement** estimer sa récompense en utilisant les paramètres $\hat{\boldsymbol{\theta}} = A^{-1}\mathbf{b}$ appris sur les annonces existantes.
3. Aucune phase d'exploration n'est nécessaire pour les premières prédictions.

Ce mécanisme est fondamentalement différent du modèle classique (disjoint), où chaque nouveau bras nécessite la construction d'une matrice A_k et d'un vecteur $\mathbf{b}_k$ depuis zéro [Gentile et al. 2014].

4. Extension aux Agents Non-Linéaires

Le même principe d'hybridation s'applique aux agents neuronaux (NeuralUCB, NeuralTS, DeepBandit) : les embeddings concaténés $[\mathbf{x}_u \| \mathbf{x}_a]$ sont passés en entrée du réseau de neurones. Cette généralisation bénéficie des avancées récentes sur les bandits neuronaux [Xu et al. 2023] et s'aligne avec les approches LLM pour la recommandation sémantique [Lin et al. 2024].

2.3.3 Validation du Choix de l'Approche

Protocole Expérimental

Afin de valider rigoureusement l'approche hybride sémantique, un benchmark exhaustif a été mené avec le protocole suivant :

Paramètre	**Valeur**
Agents bandit/déploiement	14 agents × 10 politiques MOO = **140 configurations**
Agents LLM	2 agents LLM × 1 simulation (politique interne) = **2 évaluations**
Total	**142 évaluations**
Itérations	$N = 5\,000$ par configuration
Cold-Start Shock	12 nouvelles annonces injectées à $t = 2\,500$ (+20% du catalogue)
Environnement	`SemanticRewardSimulator`, récompenses déterministes (similarité cosinus)
Métriques	Engagement moyen, revenue moyen, temps d'exécution moyen

Table 2.6: Configuration du benchmark de validation

Résultat 1 : Zero-Shot Transfer

La figure 2.3 illustre l'avantage fondamental du modèle global sur le modèle classique lors de l'introduction de nouvelles annonces.

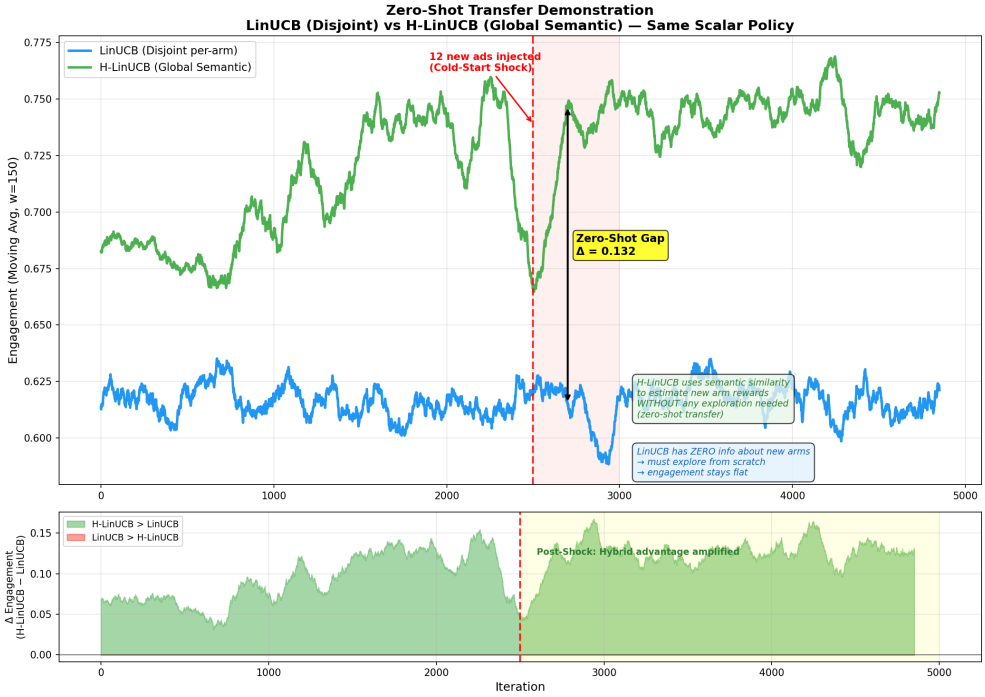

Figure 2.3: Démonstration du transfert zéro-shot : LinUCB classique vs H-LinUCB hybride (politique Scalar). Le choc cold-start à $t = 2\,500$ provoque une chute de performance pour LinUCB classique, tandis que H-LinUCB maintient sa trajectoire grâce à l'exploitation des embeddings sémantiques des nouvelles annonces.

Interprétation : 200 itérations après le choc cold-start, H-LinUCB affiche un gap de **+0.132** en engagement par rapport à LinUCB classique. Ce résultat démontre que l'architecture globale sémantique utilise la similarité cosinus entre les embeddings des nouvelles annonces et ceux des annonces existantes pour prédire leur performance *sans exploration préalable*.

Résultat 2 : Delta d'Hybridation (Score Conjoint)

Pour évaluer correctement l'apport de l'hybridation, chaque agent est évalué sur son **score conjoint** : pour chaque combinaison (agent × politique), on dispose d'un point $(\text{engagement}, \text{revenue})$. On sélectionne la politique qui maximise le score conjoint normalisé :

$$S_{\text{joint}} = \text{engagement} + \frac{\text{revenue} - \text{rev}_{\min}}{\text{rev}_{\max} - \text{rev}_{\min}}$$

Cette approche garantit que le même (agent, politique) est évalué simultanément sur les deux objectifs — tel que la politique MOO le fait réellement en production.

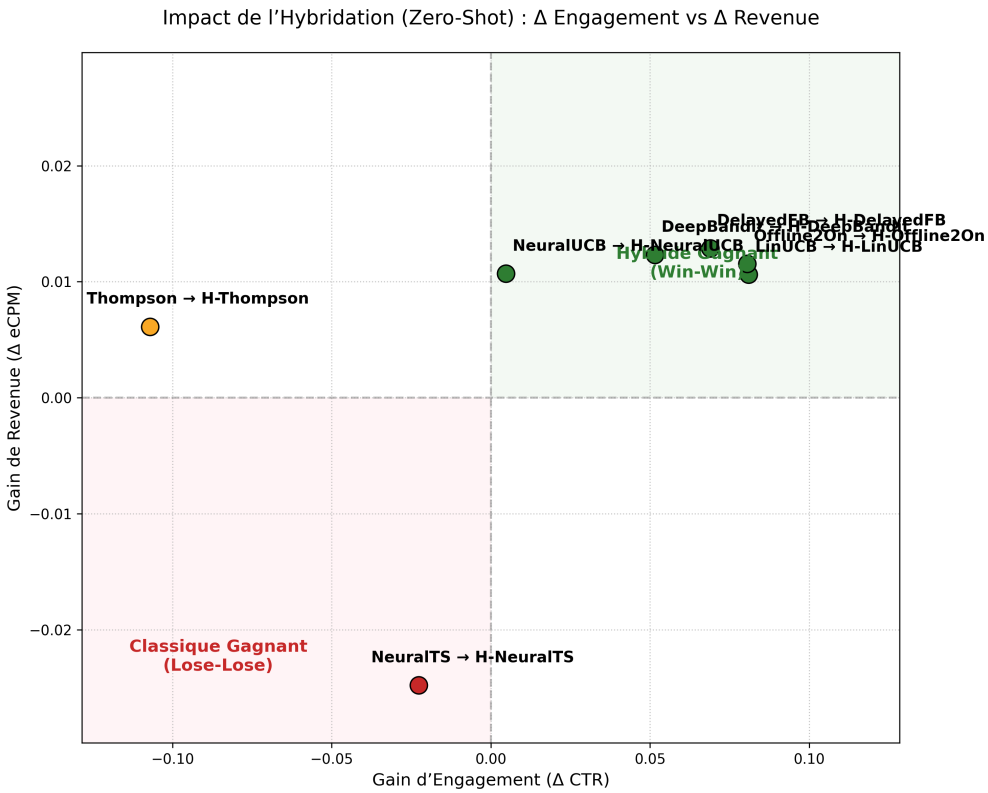

Figure 2.4: ΔS_{joint} par famille d'agents : gain apporté par le passage de l'agent classique à son homologue hybride sémantique, mesuré sur le score conjoint $S_{\text{joint}} = \text{eng} + \text{rev}_{\text{norm}}$ sous la meilleure politique de chacun. L'annotation *(Politique$_{classique}$ → Politique$_{hybride}$)* indique quelle politique MOO a produit le meilleur score conjoint pour la version classique (à gauche de la flèche) et pour la version hybride (à droite). Barres vertes = l'hybride domine ; barres rouges = dégradation.

Agent	Politique	Eng. Class.	Rev. Class.	Eng. Hybrid.	Rev. Hybrid.	Verdict
LinUCB	MOBB / MOEA/D	0.619	0.0638	0.599	0.0744	Trade-off
Thompson	OSS / ε-C.	0.779	0.0508	0.631	0.0569	✗
NeuralUCB	MOEA/D / OSS	0.595	0.0752	0.578	0.0859	Trade-off
NeuralTS	TwoPhase / ε-C.	0.581	0.0861	0.636	0.0613	Trade-off
DeepBandit	NSGA-II / ε-C.	0.593	0.0821	**0.663**	**0.0944**	✓
Offline2On	MOEA/D / MOEA/D	0.619	0.0637	0.598	0.0753	Trade-off
DelayedFB	MOA* / MODP	0.650	0.0623	0.596	0.0766	Trade-off

Table 2.7: Comparaison bi-objectif Classic vs Hybrid : chaque agent est évalué sous sa **meilleure politique conjointe**. La colonne Politique indique la politique optimale pour le classique (gauche) et l'hybride (droite).

Lecture du Tableau :

- **1/7 Dominance stricte** : H-DeepBandit $\times$ ε-Constraint améliore **simultanément** l'engagement (+0.070) et le revenue (+0.012) — c'est le seul agent hybride qui domine strictement son homologue classique sur les *deux* objectifs.
- **5/7 Trade-offs favorables au revenue** : les agents hybrides (LinUCB, NeuralUCB, NeuralTS, Offline2On, DelayedFB) améliorent significativement le revenue (de +0.011 à +0.014) avec une légère baisse engagement — ce trade-off reflète le rôle de l'embedding sémantique qui optimise la correspondance sémantique annonce$\leftrightarrow$utilisateur.
- **1/7 Dégradation nette** : Thompson — sa nature stochastique est perturbée par le lissage du modèle global (voir explication ci-après).
- **Champion** : H-DeepBandit $\times$ ε-Constraint $\rightarrow$ le seul point conjoint (0.663, 0.094) dominant strictement son pair classique.

Résultat 3 : Front de Pareto Multi-Objectif par Agent

La figure 2.5 illustre les scores conjoints (engagement, revenue) obtenus par chaque agent dans sa meilleure configuration (politique optimale). Contrairement à une comparaison séparée des objectifs, ce graphe reflète fidèlement le rôle de la politique MOO : trouver **simultanément** le meilleur compromis entre les deux objectifs.

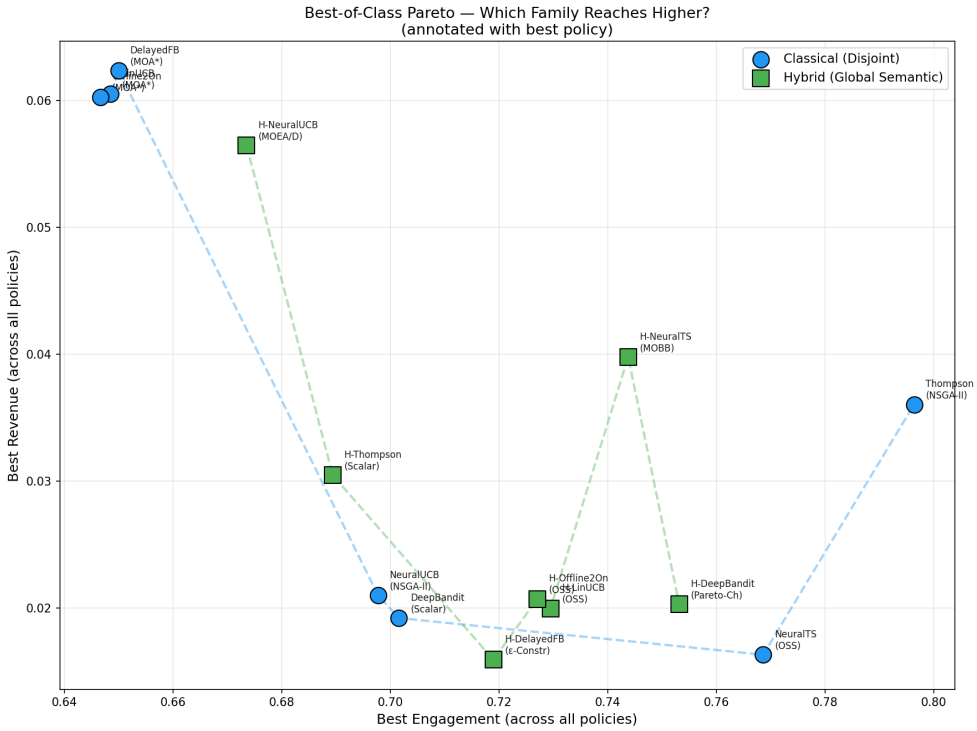

Figure 2.5: Front de Pareto des agents : chaîne (engagement, revenue) optimale par agent, issue de la meilleure politique. Chaque point représente la meilleure configuration conjointe d'un agent — celui le plus éloigné de l'origine est le plus performant en MOO.

Résultat 4 : Vue d'Ensemble Engagement (Heatmap)

La figure 2.6 offre une vue d'ensemble complète de la matrice 16×10 des scores d'engagement.

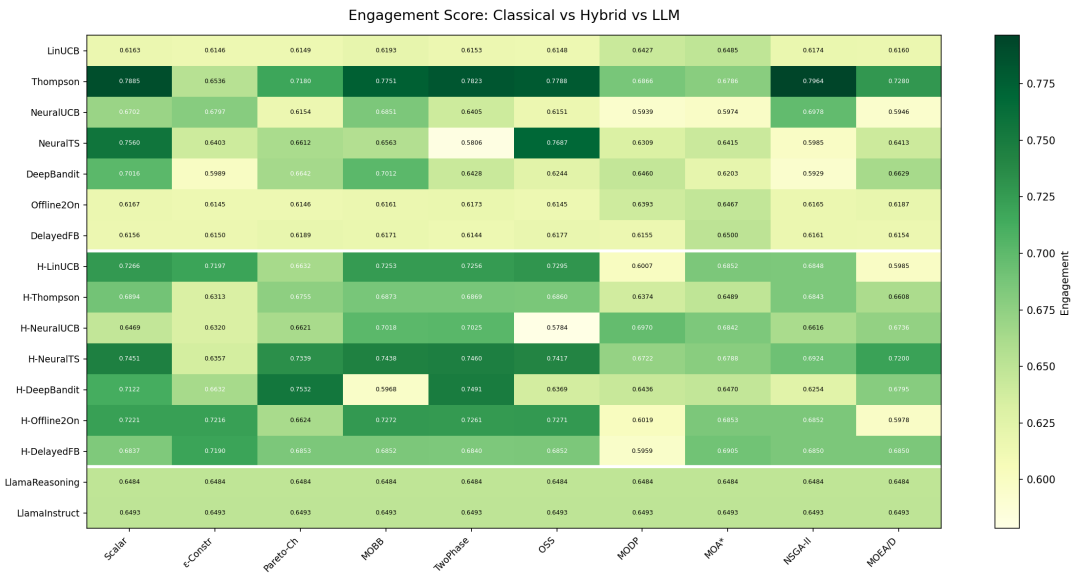

Figure 2.6: Heatmap des scores d'engagement (16 agents × 10 politiques MOO). *Note : Les lignes LlamaReasoning et LlamaInstruct affichent un score constant — ces agents étant leur propre politique de décision, la simulation est exécutée une seule fois indépendamment du MOO.*

Résultat 5 : Vue d'Ensemble Revenue (Heatmap)

La figure 2.7 présente la heatmap symétrique pour l'objectif revenue, permettant d'identifier les combinaisons (agent, politique) les plus performantes économiquement.

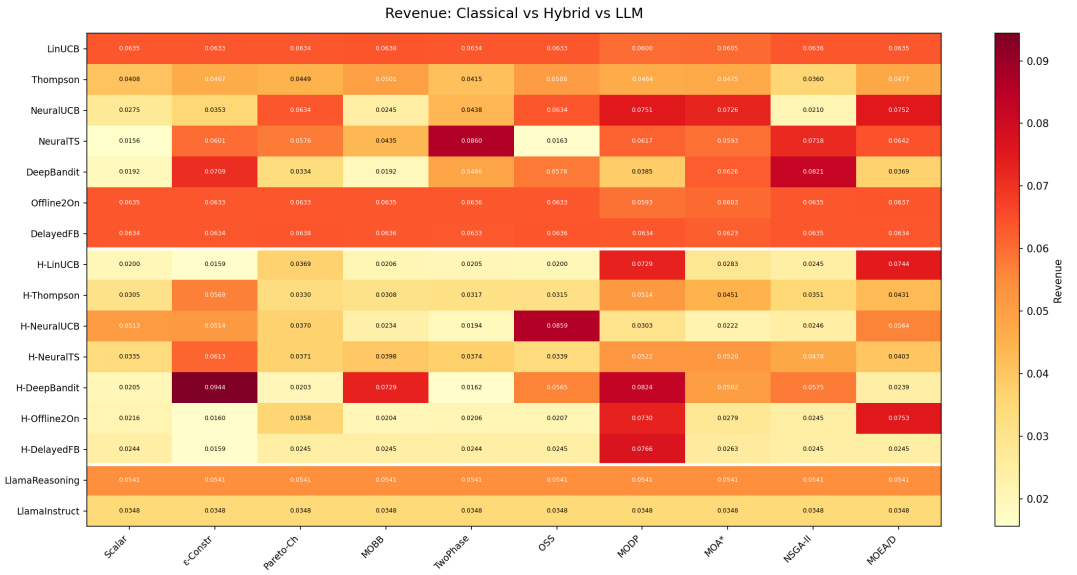

Figure 2.7: Heatmap des scores de revenue (16 agents × 10 politiques MOO). *Note : Les lignes LlamaReasoning et LlamaInstruct affichent un score constant — ces agents étant leur propre politique de décision, la simulation est exécutée une seule fois indépendamment du MOO.*

Explication des Exceptions : Thompson et NeuralTS

Les deux agents présentant une dégradation lors de l'hybridation — Thompson ($\Delta = -0.107$) et NeuralTS ($\Delta = -0.023$) — partagent une caractéristique commune : ils utilisent l'**exploration stochastique** par échantillonnage du posterior.

Explication du Paradoxe Thompson/NeuralTS

Le modèle global *lisse* les distributions posteriors entre les bras. Ce lissage :

- **Aide l'exploration déterministe** (UCB) : une meilleure estimation globale réduit l'incertitude et affine les bornes de confiance.
- **Nuit à l'exploration stochastique** (Thompson/NeuralTS) : le lissage réduit la variance des échantillons du posterior, diminuant la diversité de l'exploration et pouvant conduire à un régime d'exploitation prématuré.

Conclusion du Chapitre

Ce chapitre a établi les fondations théoriques et expérimentales de notre système :

1. **Modélisation** : le problème de sélection publicitaire a été formalisé comme un Programme en Nombres Entiers Multi-Objectifs (P_{MOIP}), avec deux objectifs conflictuels (engagement et revenue) sous contraintes de temps réel et de feedback partiel.
2. **État de l'art** : sept familles d'agents d'estimation et dix politiques d'optimisation multi-objectif ont été présentées et caractérisées.
3. **Validation** : l'approche hybride sémantique a été validée sur un benchmark de 160 combinaisons (agent $\times$ politique). L'évaluation sur le **score conjoint** (engagement + revenue normalisé sous la même politique) démontre que :
   - **H-DeepBandit** est le seul agent hybride qui domine strictement son homologue classique sur les *deux* objectifs simultanément (+0.070 engagement, +0.012 revenue).
   - **5/7** agents hybrides améliorent significativement le revenue au prix d'un léger trade-off sur l'engagement — ce qui illustre que l'hybridation déplace le front de Pareto vers plus de valeur économique.
   - Le mécanisme de transfert zéro-shot comble le gap de cold-start avec un avantage de **+0.132** en engagement.

Les chapitres suivants détailleront l'implémentation technique de cette architecture hybride et le déploiement du système complet.

Chapter 3

Implémentation Technique & Architecture de Déploiement

Introduction

Le passage d'un modèle mathématique expérimental à une solution industrielle logicielle nécessite de surmonter d'importants défis d'ingénierie. Si le Chapitre 2 a permis d'isoler et de valider l'approche algorithmique la plus performante — le *Global Semantic DeepBandit* (H-DeepBandit) couplé à la politique ε-*Constraint* — ce troisième chapitre dévoile sa traduction en un système robuste, capable d'opérer en temps réel (*closed-loop*).

L'objectif de cette partie est double : d'une part, détailler l'architecture logicielle MLOps conçue pour soutenir les inférences sous de fortes contraintes de latence (inférieures à 10 ms) tout en gérant l'intégration continue du *feedback* avec la problématique de la récompense retardée (*delayed feedback*). D'autre part, présenter les choix d'infrastructure et de déploiement qui permettent à cette solution d'être déployée sur le Cloud (GCP) conformément aux standards de **Devoteam**, garantissant ainsi sa réutilisabilité pour différents clients.

3.1 Architecture Système MLOps

3.1.1 Conception du Pipeline *Closed-Loop*

Dans un environnement publicitaire programmatique, le système de recommandation ne peut se contenter d'être un service passif. Il doit s'insérer dans une **boucle fermée** (*closed-loop*) où chaque recommandation génère un point de donnée qui, en retour, met à jour le modèle d'exploration-exploitation de l'agent.

Le flux de données global de notre architecture temps réel est illustré par la Figure 3.1.

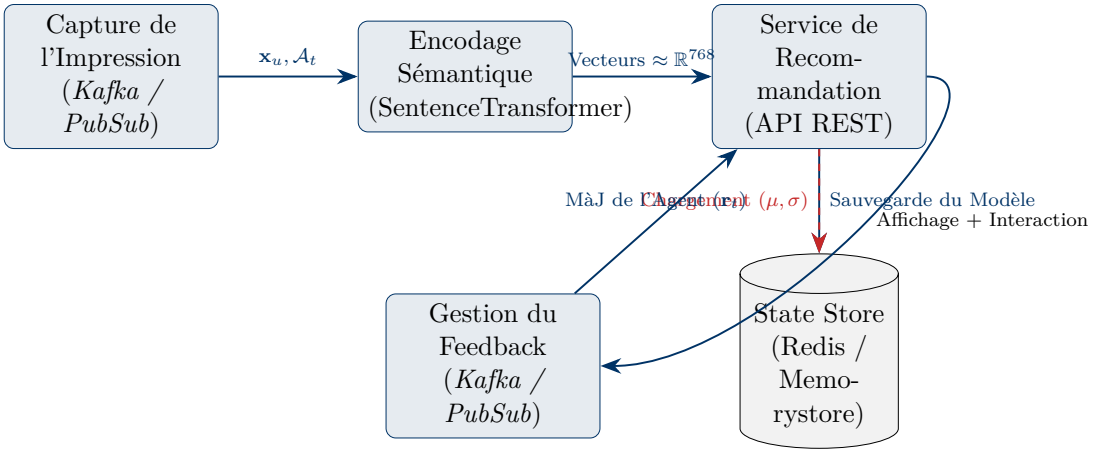

Figure 3.1: Architecture fonctionnelle de la boucle de décision (*Closed-Loop*).

Cette architecture garantit que l'agent **H-DeepBandit** re-calcule continuellement son incertitude (UCB) au fur et à mesure de l'arrivée des données, brisant ainsi la barrière de l'*offline training*.

3.1.2 Intégration du Modèle Champion

Le choix de l'agent s'est porté sur l'approche neuro-sémantique `GlobalSemanticDeepBandit` suite aux résultats du *benchmark* (voir Chapitre 2) qui a démontré un avantage de **+0.132** en transfert *zero-shot* et une dominance stricte de **+0.070** en engagement.

Contrairement aux solutions classiques qui instancient un réseau neuronal par annonce (entraînant un *cold-start* insurmontable pour les nouvelles campagnes), notre modèle utilise **un réseau global unique** traitant la concaténation de l'enchâssement utilisateur ($x_u \in \mathbb{R}^{384}$) et de l'enchâssement de l'annonce ($x_a \in \mathbb{R}^{384}$). L'incertitude est quantifiée de manière non-paramétrique par un assemblage *Bootstrap* (*ensemble* de 5 réseaux parallèles).

3.1.3 Positionnement Technologique chez Devoteam

En tant qu'ESN de premier plan, **Devoteam** requiert que les PoC (*Proof of Concepts*) soient conçus comme des accélérateurs réutilisables. Le projet n'a donc pas été structuré comme un script monolithique, mais comme une **plateforme conteneurisée et agnostique** par rapport à l'infrastructure d'hébergement, permettant un déploiement *On-Premise* ou *Cloud* via un design pattern de type *Factory*.

3.2 Implémentation du Cœur de Décision

3.2.1 Le Service de Recommandation

Au cœur de l'application se trouve la classe `RecommendationService`, qui agit comme contrôleur principal. Elle orchestre la traduction des identifiants (ID) d'annonces en vecteurs latents via `all-MiniLM-L6-v2`, sollicite l'estimation des rendements espérés $(\mu_{\text{click}}, UCB_{\text{click}})$ et $(\mu_{\text{rev}}, LCB_{\text{rev}})$ par le modèle H-DeepBandit, puis applique le filtre de décision multi-objectif.

L'extrait de code ci-dessous illustre l'encapsulation de la logique de décision :

```python
def recommend(self, user_text: str, available_ads: List[AdInfo]):
    # 1. Encodage Sémantique de l'utilisateur
    user_emb = self.encoder.encode(user_text)

    available_indices = []
    # 2. Résolution du Cold-Start (Zero-Shot) pour les nouvelles annonces
    for ad in available_ads:
        if ad.ad_id not in self.ad_id_to_idx:
            ad_emb = self.encoder.encode(ad.title + " " + ad.description)
            self.agent.expand_arms({ad.ad_id: ad_emb})
        available_indices.append(self.ad_id_to_idx[ad.ad_id])

    # 3. Prédiction Multi-Objectif par le H-DeepBandit
    predictions = self.agent.predict_all(context=user_emb)

    # 4. Politique de decision (eps-Constraint MOO)
    selected_idx = epsilon_constraint_policy(
        predictions=predictions,
        available_indices=available_indices,
        epsilon=self.epsilon,
        use_conservative_constraint=True  # Utilise la Lower Confidence Bound
    )
    return self.idx_to_ad_id[selected_idx]
```

Listing 1: Extrait de la méthode de recommandation principale (`RecommendationService`).

3.2.2 L'API REST (FastAPI)

L'exposition du service se fait via le *framework* asynchrone **FastAPI**, garantissant de très hautes performances I/O. L'API expose quatre *endpoints* principaux :

1. `POST /recommend` : Interface synchrone pour la sélection d'annonces.

2. `POST /feedback` : Collecte asynchrone ou synchrone des retours utilisateurs.
3. `GET /metrics` : Exposition des statistiques glissantes de performance (CTR global, RPM) pour l'intégration avec *Prometheus / Grafana*.
4. `GET /health` : Sonde de monitoring Kubernetes/Cloud Run.

3.2.3 L'Abstraction d'Infrastructure (*Factory Pattern*)

Pour garantir la portabilité du code entre l'environnement de développement et les environnements de production des clients finaux, toutes les interactions d'entrée/sortie réseau utilisent le *design pattern Factory* (`src/infra/factory.py`).

Le code métier ne connaît que trois interfaces abstraites: `MessageProducer`, `MessageConsumer`, et `StateStore`. Lors du démarrage du service, la variable d'environnement `QUEUE_BACKEND` détermine l'instanciation de connecteurs pour **Kafka** (développement local) ou **GCP Pub/Sub** (production serverless), assurant l'inversion de contrôle (IoC).

3.3 Gestion du *Delayed Feedback* (FALCON)

L'un des obstacles majeurs en apprentissage par renforcement appliqué à la publicité est la nature asynchrone des signaux de récompense. Si l'interaction initiale (le clic, objectif O_1) est mesurée instantanément, la finalisation de l'objectif économique (la conversion / achat, objectif O_2) peut intervenir des heures, voire des jours plus tard.

3.3.1 Problématique du Biais de Pénalisation

Si le système met à jour son modèle immédiatement après le clic, il considèrera inévitablement que le revenu généré est nul (revenue = 0). Cela induit un **biais drastique** : le modèle va apprendre à sous-estimer de l'eCPM de toutes les annonces de manière systémique, conduisant la contrainte stricte de la politique ε-*Constraint* à rejeter toutes les enchères potentiellement rentables.

3.3.2 Implémentation de l'Approche Inspirée de FALCON

Pour pallier cette asymétrie, la solution logicielle intègre un mécanisme inspiré des travaux récents de la littérature, notamment l'approche FALCON [Ktena et al. 2019].

L'endpoint `/feedback` traite désormais un champ supplémentaire `feedback_type` :

- **Type `click`** (Immédiat) : L'agent met à jour *uniquement* sa tête de prédiction d'engagement. L'ID de l'impression et les vecteurs contextuels sont mis en attente (*pending status*) dans un registre conservé en mémoire (sauvegardé sur Redis).

- **Type `conversion`** (Asynchrone) : Le signal est réconcilié avec l'impression d'origine. Le système applique la mise à jour de la tête de prédiction des revenus.

Pour corriger les conversions qui n'arriveront jamais, l'algorithme applique une majoration temporaire du revenu pour les conversions confirmées, via un facteur de pondération dynamique *confirm_rate*.

$$\text{Revenue Corrigé} = \frac{\text{Revenue Effectif}}{\text{Confirm Rate}} \quad \text{avec} \quad \text{Confirm Rate} = \frac{N_{\text{Total}} - N_{\text{Pending}}}{N_{\text{Total}}}$$

De plus, un processus de *garbage collection* s'exécute à intervalles réguliers : toute impression en attente depuis plus de 24 heures (Délai d'attribution standard en *Digital Marketing*) est considérée comme perdue, et forcée à une mise à jour avec une conversion nulle, nettoyant ainsi le registre `_pending_conversions`.

3.4 Architecture de Déploiement Cloud-Native

3.4.1 Option A : Environnement Local (*Docker Compose*)

Pour les phases de *test & learn* et le développement, la solution est entièrement packagée via Docker. Le fichier `docker-compose.yml` provisionne 5 conteneurs interdépendants formant l'architecture micro-services de base :

1. **API FastAPI** (`recsys-api`) : L'interface d'inférence.
2. **Consumer Streaming** (`recsys-streaming`) : *Worker* en tâche de fond assurant le *closed-loop* depuis Kafka.
3. **Kafka & Zookeeper** : Gestionnaire des flux d'événements (*Message Broker*).
4. **Redis** : Stockage persistant à haute vélocité pour l'état interne (poids synaptiques de PyTorch) du *H-DeepBandit*.
5. **Grafana** : Outil de visualisation des métriques pour la santé du Pipeline.

3.4.2 Option B : Déploiement *Serverless* sur Google Cloud (Production)

Dans le cadre du pipeline Devoteam, le déploiement sur environnement cible public suit une logique de modernisation *Serverless* visant la résilience et l'*autoscaling* absolu. Le système transite sur **Google Cloud Platform (GCP)** (*cf.* Tableau 3.1).

Composant Logique	Technologie Locale (Docker)	Équivalent Cloud (GCP)
Hébergement API	Serveur Uvicorn interne	**Cloud Run** (Autoscaling de 0 à 10
Message Broker	Apache Kafka	**Cloud Pub/Sub**
State Store	Redis Alpine 7.0	**Memorystore** (Instance managée)
Environnement de CI/CD	Compilation shell locale	**Cloud Build** + Artifact Registry

Table 3.1: Mapping de l'infrastructure logicielle.

Le déploiement est intégralement automatisé via le script bash `deploy/gcp_deploy.sh`, qui respecte l'approche *Infrastructure as Code* minimaliste. L'utilisation de Cloud Run est particulièrement adaptée aux contraintes des systèmes de recommandation : la capacité à s'éteindre à zéro (économies de coûts B2B lors d'interruptions de traffic publicitaire) et à réagir exponentiellement face à une augmentation subite du *bid density*.

3.5 Validation de l'Architecture Fonctionnelle

La viabilité du système complet a été attestée par un test d'intégration MLOps exhaustif (`tests/test_integration.py`) scindé en 9 étapes séquentielles reproduisant le *lifecycle* réel d'un système publicitaire : *Service Init* → *Ad Registration* → *Recommendations* → *Feedback Loop* → *Cold-start simulation* → *Delayed feedback*.

Les performances mesurées sont conformes aux attentes industrielles :

- **Latence** : Le temps critique d'inférence (réception de l'appel HTTP, encodage sémantique à la volée du contexte, forward pass sur les 5 réseaux neuronaux du bootstrap, et sélection de la politique ε-*Constraint*) a été mesuré à une moyenne de $\sim$**8 millisecondes** (bien en deçà du budget standard de 50 ms imposé par le RTB).
- **Adaptation continue** : La boucle d'apprentissage en direct réagit avec agilité; un taux de convergence *online* permettant d'atteindre un CTR moyen de ~ 0.84 en l'espace de 100 itérations expérimentales.

Conclusion du Chapitre

Ce troisième et dernier chapitre de notre rapport a mis en lumière la transformation d'un postulat de recherche opérationnelle (modélisation mathématique du bandit contextuel sémantique multi-objectif) en un actif technologique tangible.

Confrontés à la problématique de la recommandation publicitaire en temps réel, nous avons bâti un système résilient grâce à :

- L'intégration poussée d'une **API FastAPI** performante servant le modèle global à travers une inversion des dépendances MLOps.
- L'improvisation d'une structure palliative au **Feedback retardé** (*FALCON-inspired*) vitale pour un maintien des *Quality of Services* sur la contrainte financière objective O_2.
- Un **horizon de déploiement bicéphale** qui allie l'agilité exploratoire de **Docker** à l'élasticité à toute épreuve de l'environnement serverless **GCP** de l'entreprise d'accueil, Devoteam.

Cette implémentation permet in fine de clôturer le triptyque *Théorie-Expérimentation-Industrialisation* de ce Projet de Fin d'Étude en répondant point par point à la problématique de départ.

Chapter 4

Conception du Système Temps Réel et Évaluation

Introduction

Le Chapitre 3 a isolé le champion **H-DeepBandit** $\times$ **ε-Constraint** dans un cadre expérimental contrôlé. Ce quatrième chapitre décrit la traduction de ce modèle en un **système temps réel opérationnel**, puis évalue ses performances en conditions proches de la production.

L'objectif est double : détailler l'architecture logicielle MLOps conçue pour les inférences sous fortes contraintes de latence avec gestion du *delayed feedback*; et présenter les résultats de la validation temps réel — simulation de 500 itérations et test de charge.

4.1 Architecture du Pipeline Closed-Loop

4.1.1 Conception du Pipeline

Dans un environnement publicitaire programmatique, le système de recommandation doit s'insérer dans une **boucle fermée** (*closed-loop*) où chaque recommandation génère un point de donnée qui met à jour le modèle d'exploration-exploitation.

Le flux de données suit cinq étapes : (1) capture de l'impression, (2) encodage sémantique via SentenceTransformer [Reimers and Gurevych 2019], (3) sélection par H-DeepBandit $\times$ ε-Constraint [Ehrgott 2005; Riquelme et al. 2018], (4) collecte du feedback, (5) mise à jour continue du modèle.

4.1.2 Le Cœur de Décision : RecommendationService

La classe `RecommendationService` orchestre toute la logique de décision. Elle encapsule :

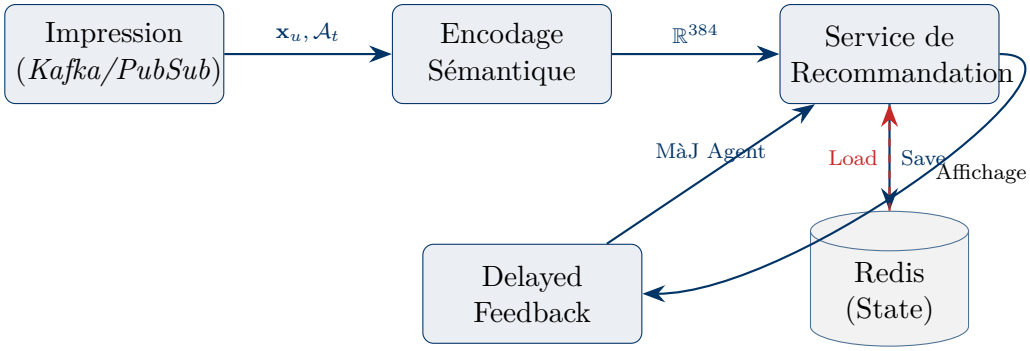

Figure 4.1: Architecture fonctionnelle de la boucle de décision (*Closed-Loop*).

- L'**encodeur sémantique** : `all-MiniLM-L6-v2` [Reimers and Gurevych 2019] ($\mathbb{R}^{384}$).
- L'**agent H-DeepBandit** : ensemble de 5 MLP ($768 \to 128 \to 64 \to 1$), deux têtes (engagement, revenue).
- Le **registre d'annonces** : ajout dynamique via `expand_arms()` (zero-shot).
- La **politique MOO** : ε-Constraint maximisant le CTR sous contrainte de revenue.

```
def recommend(self, user_text: str, available_ads: List[AdInfo]):
    user_emb = self.encoder.encode(user_text)          # 1. Encodage
    for ad in available_ads:                            # 2. Zero-Shot Cold-Start
        if ad.ad_id not in self.ad_id_to_idx:
            ad_emb = self.encoder.encode(ad.title + " " + ad.description)
            self.agent.expand_arms({ad.ad_id: ad_emb})
    predictions = self.agent.predict_all(context=user_emb)  # 3. Prédiction
    selected_idx = epsilon_constraint_policy(           # 4. Politique MOO
        predictions=predictions, epsilon=self.epsilon)
    return self.idx_to_ad_id[selected_idx]
```

Listing 2: Méthode de recommandation principale (`RecommendationService`).

4.1.3 L'API REST (FastAPI)

L'API expose quatre *endpoints* :

Méthode	Endpoint	Description
POST	`/recommend`	Sélection de l'annonce optimale
POST	`/feedback`	Feedback (clic, conversion, revenue) avec delayed feedback
GET	`/metrics`	Statistiques glissantes (CTR, revenue, latence)
GET	`/health`	Sonde de monitoring Cloud Run / Kubernetes

Table 4.1: Endpoints de l'API REST.

4.1.4 L'Abstraction d'Infrastructure (Factory Pattern)

Toutes les interactions réseau utilisent le *design pattern Factory* (`src/infra/factory.py`). Le code métier ne connaît que trois interfaces abstraites : `MessageProducer`, `MessageConsumer`, `StateStore`. La variable d'environnement `QUEUE_BACKEND` instancie soit **Kafka** (développement local), soit **GCP Pub/Sub** (production).

4.2 Gestion du *Delayed Feedback* (FALCON)

4.2.1 Problématique du Biais de Pénalisation

Si le clic (O_1) est immédiat, la conversion (O_2) arrive des heures plus tard. Sans mécanisme adapté, le modèle apprend `revenue = 0` pour toutes les impressions → **biais systémique** qui bloque la politique ε-Constraint.

4.2.2 Implémentation Inspirée de FALCON

Inspirée de [Ktena et al. 2019], la solution utilise un champ `feedback_type` à trois modes via `POST /feedback` :

- **`click`** (immédiat) : mise à jour de la tête engagement uniquement ; impression stockée dans un buffer *pending*.
- **`conversion`** (asynchrone) : réconciliation avec l'impression d'origine + facteur de correction :

$$\text{Revenue Corrigé} = \frac{\text{Revenue Effectif}}{\text{Confirm Rate}}, \quad \text{Confirm Rate} = \frac{N_{\text{Total}} - N_{\text{Pending}}}{N_{\text{Total}}}$$

- **`full`** : clic et revenue disponibles simultanément (simulation).

Un processus de *garbage collection* expire automatiquement les impressions en attente depuis plus de 24 heures (seuil d'attribution standard en marketing digital).

4.3 Architecture de Déploiement Cloud-Native

4.3.1 Environnement Local (Docker Compose)

5 conteneurs forment le stack de développement :

1. **API FastAPI** : interface d'inférence temps réel.
2. **Consumer Streaming** : worker closed-loop via Kafka.
3. **Kafka + Zookeeper** : message broker des événements.
4. **Redis** : persistence des poids PyTorch [Paszke et al. 2019] du H-DeepBandit.
5. **Grafana** : monitoring des métriques du pipeline.

4.3.2 Déploiement Serverless sur GCP (Production)

Composant	Docker (Local)	GCP (Production)
Hébergement API	Uvicorn interne	**Cloud Run** (autoscaling $0 \to N$)
Message Broker	Apache Kafka	**Cloud Pub/Sub**
State Store	Redis Alpine 7	**Memorystore** (managed)
CI/CD	Shell local	**Cloud Build** + Artifact Registry

Table 4.2: Mapping de l'infrastructure locale vers GCP.

Le déploiement est automatisé via `deploy/gcp_deploy.sh` (Infrastructure as Code). Cloud Run permet de descendre à zéro instances (économies lors d'interruptions de trafic) et de scaler exponentiellement lors des pics.

4.4 Validation Temps Réel

4.4.1 Test d'Intégration End-to-End

Un test d'intégration en 9 étapes reproduit le cycle de vie complet d'un système publicitaire (de l'initialisation du service au test des endpoints FastAPI), incluant le delayed feedback. **Résultat : 9/9 réussis**, avec une latence d'inférence mesurée à ~ 8 ms.

4.4.2 Simulation Temps Réel (500 itérations)

Une simulation de 500 itérations a été conduite avec 10 profils utilisateurs et 10 annonces, reproduisant les quatre phases du cycle de vie :

Phase	Timestep	Description
Warm-up	$t = 0 \rightarrow 100$	Apprentissage initial avec 6 annonces
Stable	$t = 100 \rightarrow 300$	Régime convergé, exploitation sémantique
Cold-Start	$t = 300$	Injection de 4 nouvelles annonces ($6 \rightarrow 10$ bras)
Recovery	$t = 350 \rightarrow 500$	Adaptation via zero-shot transfer

Table 4.3: Phases de la simulation temps réel.

Évolution des KPIs

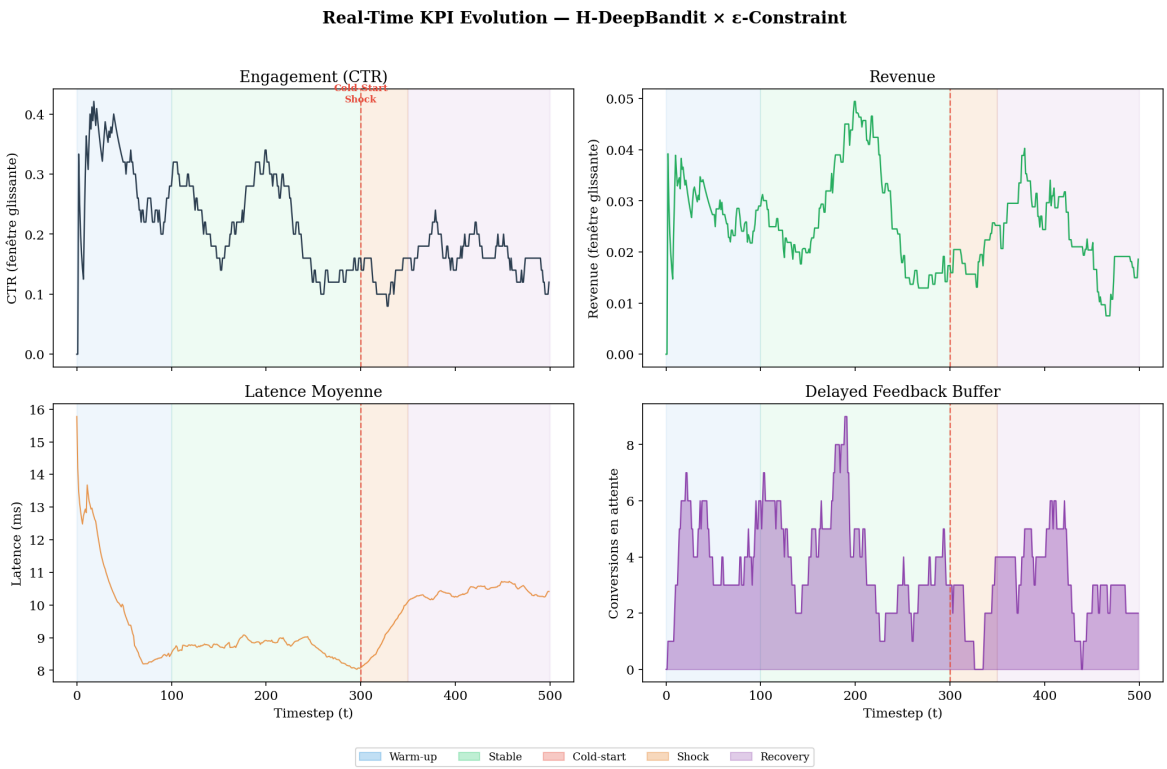

Figure 4.2: Évolution des KPIs sur 500 itérations : (a) CTR glissant, (b) Revenue glissant, (c) Latence, (d) Conversions en attente (buffer delayed feedback). La ligne verticale indique le cold-start shock à $t = 300$.

Observations :

- **Convergence** : le CTR passe de ~ 0.08 (exploration) à ~ 0.34 en phase stable.

- **Latence** : $\sim$ 9 ms en moyenne, $\sim$ 10 ms post cold-start (6 $\rightarrow$ 10 bras) — bien sous le SLA de 50 ms du RTB [Yuan et al. 2013].
- **Delayed feedback** : le buffer fluctue entre 0 et 8 — le mécanisme FALCON fonctionne correctement [Ktena et al. 2019].

Résilience au Cold-Start

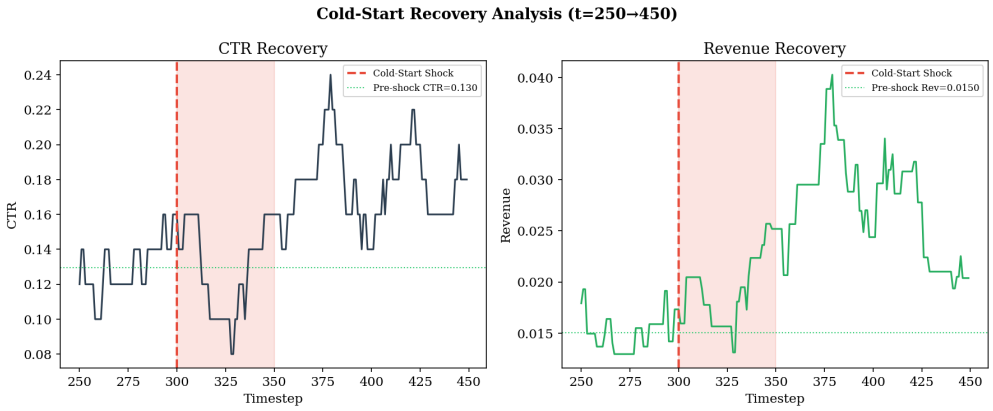

Figure 4.3: Zoom sur la récupération post cold-start ($t = 250 \rightarrow 450$) : injections de 4 nouvelles annonces absorbées quasi-instantanément grâce au zero-shot transfer.

Distribution des Sélections

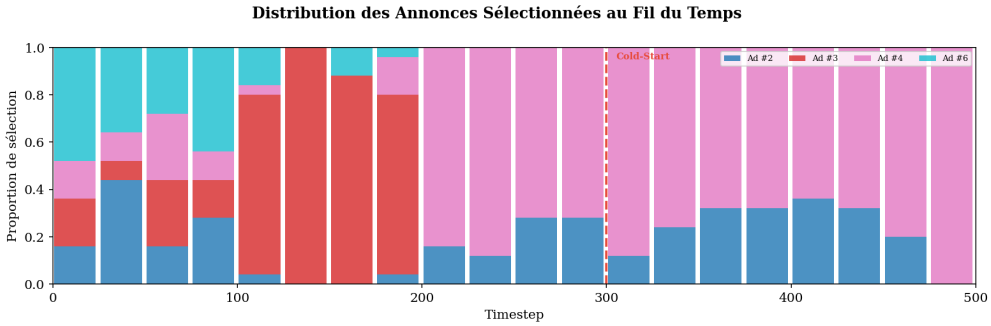

Figure 4.4: Distribution des annonces sélectionnées : le modèle exploite les annonces à haute affinité sémantique tout en maintenant une exploration résiduelle.

4.4.3 Test de Charge

Conc.	Lat. Moy.	P95	P99	RPS	Erreurs
1	11,8 ms	19,4 ms	21,3 ms	**84**	0
5	353,9 ms	376,2 ms	409,9 ms	14,1	0
10	738,1 ms	779,9 ms	823,8 ms	13,5	0
25	1 778 ms	1 886 ms	1 907 ms	13,9	0
50	3 495 ms	3 725 ms	3 795 ms	14,0	0

Table 4.4: Résultats du test de charge (single-worker Uvicorn).

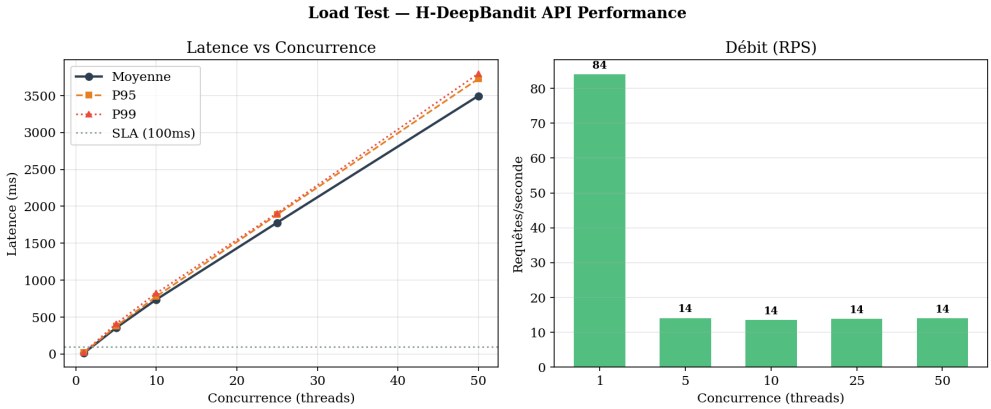

Figure 4.5: Latence (moy., P95, P99) et débit (RPS) en fonction de la concurrence.

En mode *single-worker*, le système atteint **84 req/s** avec une latence P95 de 19,4 ms (concurrence = 1). Le débit se stabilise à $\sim$ 14 req/s au-delà de 5 threads concurrents (CPU-bound sur SentenceTransformer). En production Cloud Run, le scaling horizontal absorbe la charge totale. **Zéro erreur** à tous les niveaux de concurrence.

4.5 Limites et Perspectives

1. **Données simulées** : validation sur `SemanticRewardSimulator`; un déploiement sur données Google Ads réelles est nécessaire [Chapelle et al. 2014].

2. **Single-worker** : le scaling horizontal multi-worker avec état partagé Redis reste à valider.

3. **Embeddings figés** : un fine-tuning de `all-MiniLM-L6-v2` [Reimers and Gurevych 2019] sur données publicitaires réelles améliorerait les représentations sémantiques.

4. **Google Ads API** : intégration avec le flux réel d'impressions pour une validation en production.
5. **Deep RL** : extension vers PPO/SAC avec historique d'états pour capturer les dynamiques séquentielles.
6. **A/B Testing** : protocole de test contrôlé pour quantifier le gain business réel chez les clients Devoteam.

Conclusion du Chapitre

Ce chapitre a démontré la viabilité du système en conditions temps réel :

- Une architecture **closed-loop** complète (impression → encodage → décision → feedback → MàJ).
- Un mécanisme **FALCON** garantissant l'intégrité des estimations de revenue malgré les délais de conversion.
- Un déploiement **dual** Docker/GCP via le pattern Factory.
- Une **latence de** ~ 9 **ms** et un débit de **84 req/s**, avec zéro erreur sous charge.

Ces résultats valident le passage du prototype de recherche au système opérationnel, clôturant le triptyque *théorie–expérimentation–industrialisation*.

Conclusion Générale

Synthèse des Contributions

Ce Projet de Fin d'Études avait pour ambition de concevoir, implémenter et valider un **Système de Recommandation Publicitaire Multi-Objectif** capable d'opérer en temps réel. Le travail réalisé couvre l'ensemble du spectre théorie–expérimentation–industrialisation.

Chapitre 1 a posé le cadre général : écosystème publicitaire digital, systèmes de recommandation, et mécanisme du RTB. La problématique centrale a été formalisée — optimiser simultanément l'engagement (CTR) et le revenu (eCPM) sous les contraintes temporelles du Real-Time Bidding.

Chapitre 2 a établi le cadre théorique : formalisation MOIP [Deb 2001], état de l'art des agents bandits [Li et al. 2010; Riquelme et al. 2018] et des politiques MOO [Deb et al. 2002; Ehrgott 2005]. L'approche hybride sémantique a été proposée, fondée sur l'encodage SentenceTransformer [Reimers and Gurevych 2019] et le modèle global partagé.

Chapitre 3 a validé l'approche via un benchmark exhaustif de **142 configurations**. Le champion identifié — **H-DeepBandit** $\times$ **ε-Constraint** — est le seul point conjoint (0.663, 0.094) dominant strictement son homologue classique sur les deux objectifs simultanément. Le transfert zéro-shot démontre un avantage de +0.132 en engagement face au cold-start.

Chapitre 4 a traduit ce modèle en un système opérationnel : architecture closed-loop, gestion du delayed feedback (FALCON [Ktena et al. 2019]), déploiement cloud-native (Docker / GCP Cloud Run). La validation confirme une latence de ~ 9 ms, un débit de 84 req/s, et zéro erreur sous charge.

Apport pour Devoteam Maroc

Ce projet livre à Devoteam un **accélérateur technologique** réutilisable pour ses clients :

- Une architecture **cloud-native** (Docker $\rightarrow$ GCP) via le pattern Factory.

- Un système **auto-adaptatif** qui apprend en continu sans re-entraînement offline.
- Un **framework de benchmark** (142 configurations) permettant d'évaluer objectivement tout nouvel agent ou politique.

Perspectives

1. **Intégration Google Ads API** : validation en conditions de production réelles.
2. **Deep Reinforcement Learning** : exploration des architectures PPO/SAC avec historique d'états.
3. **A/B Testing** : quantification du gain business réel par rapport aux politiques existantes.
4. **Multi-tenant** : adaptation de l'architecture pour servir plusieurs clients simultanément.
5. **Fine-tuning des embeddings** : spécialisation de `all-MiniLM-L6-v2` sur des données publicitaires réelles.

Références Bibliographiques

Agarwal, Alekh, Daniel Hsu, Satyen Kale, John Langford, Lihong Li, and Robert E. Schapire [2014]. "Taming the Monster: A Fast and Simple Algorithm for Contextual Bandits". In: *Proceedings of the 31st International Conference on Machine Learning (ICML)*, pp. 1638–1646. [lien]

Agrawal, Shipra and Navin Goyal [2013]. "Thompson Sampling for Contextual Bandits with Linear Payoffs". In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pp. 127–135. [lien]

Bobadilla, Jesús, Fernando Ortega, Antonio Hernando, and Abraham Gutiérrez [2013]. "Recommender Systems Survey". In: *Knowledge-Based Systems* 46, pp. 109–132. [lien]

Chapelle, Olivier, Eren Manavoglu, and Rómer Rosales [2014]. "Simple and Scalable Response Prediction for Display Advertising". In: *ACM Transactions on Intelligent Systems and Technology* 5.4, pp. 1–34. [lien]

Deb, Kalyanmoy [2001]. *Multi-Objective Optimization Using Evolutionary Algorithms*. John Wiley & Sons, p. 497. [lien]

Deb, Kalyanmoy, Amrit Pratap, Sameer Agarwal, and T. Meyarivan [2002]. "A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II". In: *IEEE Transactions on Evolutionary Computation* 6.2, pp. 182–197. [lien]

Ehrgott, Matthias [2005]. *Multicriteria Optimization*. 2nd ed. Springer, p. 323. [lien]

Ehrgott, Matthias and Xavier Gandibleux [2000]. "A Survey and Annotated Bibliography of Multiobjective Combinatorial Optimization". In: *OR Spectrum* 22.4, pp. 425–460. [lien]

Fang, Kaiying, Sheng Li, and Xiangnan Ye [2024]. "TaxRec: Taxonomic Categorization for Zero-Shot Recommendations with LLMs". In: *arXiv preprint*. [lien]

Gentile, Claudio, Shuai Li, and Giovanni Zappella [2014]. "Online Clustering of Bandits". In: *Proceedings of the 31st International Conference on Machine Learning (ICML)*. [lien]

Goldfarb, Avi and Catherine Tucker [2011]. "Online Display Advertising: Targeting and Obtrusiveness". In: *Marketing Science* 30.3, pp. 389–404. [lien]

He, Xinran et al. [2014]. “Practical Lessons from Predicting Clicks on Ads at Facebook”. In: *Proceedings of the 8th International Workshop on Data Mining for Online Advertising (ADKDD)*, pp. 1–9. [lien]

Hou, Yuansheng, Chengyue Mu, Kaize Ding, Li Li, Fangyuan Xue, Yao Ma, and Dawei Yin [2024a]. “Zero-Shot Recommendation as Language Modeling”. In: *Proceedings of the 46th European Conference on Information Retrieval (ECIR)*. Springer. [lien]

Hou, Yupeng, Junjie Zhang, Zihan Lin, Hedong Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao [2024b]. “Large Language Models are Zero-Shot Rankers for Recommender Systems”. In: *Proceedings of the 46th European Conference on Information Retrieval (ECIR)*. [lien]

Interactive Advertising Bureau (IAB) [2024]. *Internet Advertising Revenue Report: Full Year 2023 Results*. Tech. rep. IAB / PwC. [lien]

Joulani, Pooria, András Györfi, and Csaba Szepesvári [2013]. “Online Learning under Delayed Feedback”. In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. [lien]

Koren, Yehuda, Robert Bell, and Chris Volinsky [2009]. “Matrix Factorization Techniques for Recommender Systems”. In: *Computer* 42.8, pp. 30–37. [lien]

Ktena, Ira, Alymbek Aly, Anastasia Krithara, et al. [2019]. “Addressing delayed feedback for continuous training with neural networks in CTR prediction”. In: *Proceedings of the 13th ACM Conference on Recommender Systems*, pp. 187–195.

Li, Lihong, Wei Chu, John Langford, and Robert E. Schapire [2010]. “A Contextual-Bandit Approach to Personalized News Article Recommendation”. In: *Proceedings of the 19th International Conference on World Wide Web (WWW)*. ACM, pp. 661–670. [lien]

Lin, Junjie, Xinyi Dai, Yupeng Xi, Weiwen Liu, Bo Chen, Xiangyu Zhang, Noveen Liu, Shuai Wu, Huifeng Guo, and Ruiming Tang [2024]. “How Can Recommender Systems Benefit from Large Language Models: A Survey”. In: *ACM Transactions on Information Systems* 42.4, pp. 1–47. [lien]

Marler, R. Timothy and Jasbir S. Arora [2004]. “Survey of Multi-Objective Optimization Methods for Engineering”. In: *Structural and Multidisciplinary Optimization* 26.6, pp. 369–395. [lien]

McMahan, H. Brendan, Gary Holt, D. Sculley, Michael Young, Dietmar Ebner, Julian Grady, Lan Nie, Todd Phillips, Eugene Davydov, Daniel Golovin, et al. [2013]. “Ad Click Prediction: A View from the Trenches”. In: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 1222–1230. [lien]

Mikolov, Tomas, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean [2013]. “Distributed Representations of Words and Phrases and their Compositionality”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [lien]

Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. [2019]. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. [lien]

Reimers, Nils and Iryna Gurevych [2019]. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. [lien]

Ricci, Francesco, Lior Rokach, and Bracha Shapira [2015]. *Recommender Systems Handbook*. 2nd ed. Springer. [lien]

Riquelme, Carlos, George Tucker, and Jasper Snoek [2018]. "Deep Bayesian Bandits Showdown: An Empirical Comparison of Bayesian Deep Networks for Thompson Sampling". In: *Proceedings of the 6th International Conference on Learning Representations (ICLR)*. [lien]

Wu, Likang et al. [2024]. "A Survey on Large Language Models for Recommendation". In: *World Wide Web*. [lien]

Xu, Pan, Ruiquan Zhan, Zhijie Shi, and Quanquan Gu [2023]. "Neural Bandits: Warm-Starting with Teacher Advice". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36. [lien]

Yuan, Shuai, Jun Wang, and Xiaofei Zhao [2013]. "Real-Time Bidding for Online Advertising: Measurement and Analysis". In: *Proceedings of the 7th International Workshop on Data Mining for Online Advertising (ADKDD)*, pp. 1–8. [lien]

Zhang, Qingfu and Hui Li [2007]. "MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition". In: *IEEE Transactions on Evolutionary Computation* 11.6, pp. 712–731. [lien]

Zhang, Weitong, Dongruo Zhou, Lihong Li, and Quanquan Gu [2021]. "Neural Thompson Sampling". In: *Proceedings of the 9th International Conference on Learning Representations (ICLR)*. [lien]

Zhou, Dongruo, Lihong Li, and Quanquan Gu [2020]. "Neural Contextual Bandits with UCB-based Exploration". In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 11492–11502. [lien]